

H A N D B O O K

Typst

From ZERO to HERO

Stefano Coelati Rama

Written and typeset entirely in Typst

Typst: from ZERO to HERO

A (nearly) Complete Handbook in English

This document was written *entirely* in the Typst language, with no help from any other tool. It is itself a demonstration of the language's typesetting and programming capabilities. It may contain plenty of typos (linguistic and otherwise)... appreciate the effort.

Version 1.0 — June 2026

All the examples in this handbook can be run directly in the official online editor: typst.app

The content of this handbook is released for educational use. Should this handbook become outdated or should some information be missing, I invite you to visit the [community](#) or the [docs](#) to look things up. Otherwise, like all common mortals, ask the artificial intelligence (I tried using it myself too — Claude in particular — while writing this handbook. It helped me with a few boxes, but (perhaps because Typst is too new) the code is almost always broken and has to be fixed by hand, ergo: study). This text is not for profit, Typst is an open-source project! <https://github.com/typst/typst>

You can find my contact details here: <https://stefanocoelatirama.com>

Contents

1. Preface	i
1.1. Who this is for	i
1.2. How to use this handbook	i
1.3. My typographic conventions	ii
2. What Typst is and why use it	3
2.1. Markup: what it is and how it works	3
2.2. Comparison with the most common tools	3
2.2.1. Typst vs Microsoft Word / LibreOffice	3
2.2.2. Typst vs LaTeX	4
2.3. How Typst works: the workflow	4
2.4. A very brief historical note	4
3. Installation and first document	5
3.1. Option 1: Online editor (recommended to start)	5
3.2. Option 2: Local installation	6
3.2.1. Installation on Windows	6
3.2.2. Installation on macOS	6
3.2.3. Installation on Linux	6
3.2.4. VS Code extension	6
3.2.5. Compiling from the command line	6
3.3. Structure of a .typ file	7
3.4. Your first document: "Hello, W...Typst!"	7
4. Text and basic formatting	8
4.1. Writing plain text	8
4.1.1. Paragraphs	8
4.1.2. Line break without a new paragraph	9
4.2. Bold, italic, and underline	9
4.2.1. Bold	9
4.2.2. Italic	10
4.2.3. Combining bold and italic	10
4.2.4. Strikethrough text	10
4.2.5. Inline code	10
4.2.6. Superscript and subscript	11
4.3. Special characters and how to use them	11
4.4. Typographic quotes	12
4.5. Special spaces	12
4.6. Horizontal line	13
5. Headings and document structure	13

5.1. Heading levels	13
5.2. Automatic numbering	14
5.3. The automatic table of contents with <code>#outline()</code>	14
5.4. Headings outside the TOC	15
5.5. Labels and cross-references	15
5.5.1. Assigning a label	15
5.5.2. Referencing	15
5.6. Page break	16
6. Lists	16
6.1. Bulleted list	16
6.1.1. Custom markers	17
6.2. Numbered list	17
6.2.1. Numbering style	17
6.2.2. Starting from a specific number	18
6.3. Nested lists	18
6.4. Definition list (term: definition)	19
6.5. Multi-paragraph items	19
6.6. Spacing between items	20
7. Links and references	20
7.1. Links to web pages	20
7.1.1. Basic syntax	20
7.1.2. Automatic URLs	20
7.1.3. Link without visible text (URL only)	21
7.1.4. Links to email addresses	21
7.2. Footnotes	21
7.3. Links and references summary	21
8. Images and figures	22
8.1. Inserting an image	22
8.1.1. Supported formats	22
8.1.2. The <code>fit</code> parameter	22
8.2. The <code>#figure()</code> function: figures with a caption	23
8.2.1. Components of <code>#figure()</code>	23
8.2.2. Customizing the supplement	23
8.2.3. Figure placement	23
8.3. Alignment	23
8.4. Side-by-side images	24
8.5. Image formats and optimization	24
8.5.1. SVG images for vector graphics	24
8.5.2. Images: local files only (no URLs)	24
8.5.3. Alternative text (accessibility)	25
8.5.4. Clip and zoom: showing a portion of an image	25
8.5.5. Side-by-side images with individual captions	25

8.5.6. Rotation and transformation	26
8.5.7. Image as page background	26
8.6. Practical tips	26
9. Tables	27
9.1. Your first table	27
9.2. Defining columns	27
9.3. Table headers	28
9.4. Content alignment	28
9.5. Alternating colors (zebra stripes)	29
9.6. Borders (stroke)	29
9.7. Cells spanning multiple columns/rows	29
9.8. Tables as figures	30
10. Mathematics	30
10.1. Math mode	30
10.1.1. Inline math	30
10.1.2. Block (display) math	31
10.2. Basic operators	31
10.3. Powers and subscripts	32
10.4. Fractions	33
10.5. Roots	33
10.6. Common special symbols	33
10.6.1. Greek letters	33
10.6.2. Other common symbols	34
10.7. Sums, products, limits	34
10.8. Vectors and matrices	35
10.9. Text inside formulas	35
10.10. Numbered formulas	35
10.11. Spaces in math	36
11. Code blocks	36
11.1. Inline code	36
11.2. Code block	36
11.3. Supported languages	37
11.4. Line numbers	37
11.5. Custom code style	37
11.6. Line numbers and manual highlighting	38
12. Introduction to code: the <code>\#</code> symbol	38
12.1. Two modes: Markup and Code	38
12.2. The <code>#</code> symbol in action	39
12.3. Inline expressions with <code>#</code>	39
12.4. Code blocks with <code>{ }</code>	39
12.5. Content blocks with <code>[]</code>	40
12.6. The fundamental difference: <code>{ }</code> vs <code>[]</code>	40

12.7. The basic principle: <code>#</code> for all code	41
12.8. The execution context	41
13. Data types	42
13.1. Overview of types	42
13.2. Strings	42
13.2.1. String operations	43
13.2.2. String interpolation	43
13.3. Integers (int)	43
13.4. Decimal numbers (float)	44
13.5. Booleans (bool)	44
13.6. Lengths and measures	45
13.7. Colors	45
13.8. Arrays (lists)	46
13.9. Dictionaries (dict)	47
13.10. Content	47
13.11. none and auto	48
13.12. Discovering a value's type	48
13.13. Type conversions	48
14. Variables	49
14.1. Defining a variable with <code>#let</code>	49
14.2. Using a variable	49
14.3. Why use variables?	49
14.4. Variable scope	50
14.5. Variables that hold content	50
14.6. Constants vs variables	51
14.7. Operations and calculations	51
15. Conditions: if, else if, else	51
15.1. The basic if structure	52
15.2. if ... else	52
15.3. if ... else if ... else	53
15.4. Comparison operators	53
15.5. Logical operators	53
15.6. if as an expression	54
16. Loops: for and while	55
16.1. The for loop	55
16.1.1. Iterating over an array	55
16.1.2. Iterating with range	55
16.1.3. range with step	56
16.1.4. Generating a table with for	56
16.1.5. Iterating with an index	57
16.1.6. for returns an array	57
16.2. The while loop	57

16.3. break and continue	58
16.4. Generating complex content with loops	58
17. Functions	59
17.1. Typst's built-in functions	59
17.2. Defining your own functions	59
17.2.1. Simple function	60
17.2.2. Function with a complex body	60
17.3. Named parameters and default values	60
17.4. Functions that accept content	61
17.4.1. The special syntax with a body parameter	62
17.5. Functions that return values	62
17.6. Arrow functions (closures)	63
17.7. Recursion	63
17.8. Complete example: a card template	64
18. The set rules	65
18.1. What is a set rule?	65
18.2. set text — text	65
18.3. set par — paragraphs	66
18.4. set heading — headings	67
18.5. set page — the page	67
18.6. set list and set enum	67
18.7. set table — tables	68
18.8. set figure — figures	68
18.9. set math.equation — equations	68
18.10. The scope of set rules	68
18.11. set inside functions	69
19. The show rules	70
19.1. The difference between set and show	70
19.2. Transforming bold	70
19.3. Transforming links	70
19.4. Transforming inline code	71
19.5. Selectors: .where()	71
19.6. Transforming headings	72
19.6.1. First purple section	72
19.6.2. Second purple section	72
19.7. Transforming the whole document (global show)	72
19.8. Transforming figures	73
19.9. show for text substitution	73
20. Colors and graphics	74
20.1. Color systems	74
20.1.1. RGB	74
20.1.2. Grayscale	74

20.1.3. OKLCH (modern, perceptually uniform)	74
20.1.4. Predefined colors	75
20.2. Manipulating colors	75
20.3. Gradients	76
20.3.1. Linear gradient	76
20.3.2. Radial gradient	76
20.4. Drawing basic shapes	77
20.4.1. Line	77
20.4.2. Rectangle	77
20.4.3. Circle and ellipse	77
20.5. The stroke	78
21. Boxes and blocks	79
21.1. box vs block: the difference	79
21.2. Parameters of box and block	79
21.2.1. Custom rounded corners	80
21.2.2. Detailed inset	80
21.3. Preventing page breaks	80
21.4. Positioning with place()	80
21.5. Stack: stacking elements	81
21.6. Alignment	82
22. Advanced layout	82
22.1. Headers and footers	82
22.1.1. Simple header	83
22.1.2. Header with page number	83
22.1.3. Context and document awareness	83
22.1.4. Header with current section title	84
22.2. Page numbering	84
22.2.1. Changing numbering within the document	84
22.3. Columns	84
22.3.1. Skipping a column	85
22.4. Special pages	85
22.4.1. Page with different margins	85
22.4.2. Landscape page	86
23. Grid layout with grid()	86
23.1. Basic syntax	86
23.2. Defining columns	86
23.3. Practical layout examples	87
23.3.1. Cover layout (icon + text)	87
23.3.2. Three info-column layout	87
23.3.3. Grid with spanning cells	88
23.4. The layout() function	88
24. Multiple files and modules	88

24.1.	<code>#include</code> — including content	89
24.1.1.	Advantages of <code>#include</code>	89
24.2.	<code>#import</code> — importing functions and variables	89
24.2.1.	Importing everything with <code>*</code>	90
24.2.2.	Importing with aliases	90
24.2.3.	Difference between import and include	90
24.3.	Organizing a large project	90
24.4.	Shared variables across files	91
25.	Packages	91
25.1.	Where to find packages	91
25.2.	How to import a package	92
25.3.	Main packages	92
25.3.1.	showybox — Styled boxes	92
25.3.2.	codly — Code with line numbers	92
25.3.3.	cetz — Vector drawing	92
25.3.4.	fletcher — Diagrams and arrows	93
25.3.5.	physica — Physics notation	93
25.3.6.	polylux — Presentations	93
25.3.7.	tablex — Advanced tables	93
25.3.8.	lovelace — Pseudocode and algorithms	94
25.3.9.	code1st — Code listings with references	94
25.3.10.	diagraph — Graphs with GraphViz	94
25.4.	How to choose the right version	94
26.	Complete academic document	94
26.1.	Minimal template for a thesis	95
26.2.	Numbered equations and references	97
26.3.	Bibliographic citations	97
26.4.	Academic figures and tables	98
27.	Curriculum Vitae template	99
27.1.	Two-column CV template	99
27.2.	Minimalist CV variant (one column)	102
28.	Presentations (Slides)	103
28.1.	Setup with Polylux	103
28.2.	Progressive reveal (animations)	106
28.3.	The Touying package (advanced alternative)	106
29.	Advanced topics	107
29.1.	The context system	107
29.1.1.	Counters	107
29.1.2.	Query	107
29.1.3.	State (global state)	108
29.2.	Advanced text algorithms	108
29.2.1.	Text with line-by-line control	108

29.2.2. Vertical text	108
29.3. Patterns and repetitions	108
29.4. Advanced drawing with CeTZ	109
29.5. Diagrams with Fletcher	110
Appendix A – Common Errors	110
29.6. Frequent syntax errors	110
29.7. Common error messages	111
Appendix B – Resources and Community	111
29.8. Official documentation	111
29.9. Community and support	111
29.10. Complementary tools	111
Appendix C – Fonts and Typography	112
29.11. How to specify fonts	112
29.12. Fonts available in the online editor	112
29.13. Font weights and styles	112
29.14. Typographic measures	113
29.15. Highlight and emphasis without bold	113
Appendix D – Counters and Numbering	114
29.16. Predefined counters	114
29.17. Creating a custom counter	114
29.18. Resetting a counter	115
29.19. Displaying the total number of pages	116
Appendix E – Ready-to-Use Templates	116
29.20. Template: Generic professional document	116
29.21. Template: Minimalist document	116
29.22. Template: Technical document (manual style)	117
29.23. Template: University lecture notes	117
29.24. Template: Newsletter / Bulletin	118
29.25. Template: Academic poster (A1)	119
Appendix F (Part I) – Additional Notes: Slides and Presentations	119
29.26. Overview of the tools	119
29.26.1. Beamer	120
29.26.2. R Markdown	121
29.26.3. Quarto	122
29.26.4. knitr	122
29.26.5. Polylux	123
29.26.6. Touying	124
29.27. Final comparison: which to choose?	126
Appendix F (Part II) – Additional Notes: Programming and Automation	126
29.28. The Python <code>typst</code> package	126
29.28.1. Compiling a file	127

29.28.2. Passing parameters with sys.inputs	127
29.29. Reading JSON data in the document	128
29.30. The Data → Template → PDF pattern	129
29.31. Batch production: certificate example	130
29.32. Integration with FastAPI	130
29.33. GitHub Actions: automatic reports	131
29.34. Best practices for Typst in production	132

1. Preface

This handbook was born from a simple question: **is it possible to learn Typst starting from absolute zero? (Even without ever having programmed?)** The answer is yes, and this text will prove it to you page after page.

Typst is a modern language for creating professional-quality documents: scientific articles, dissertations, presentations, CVs, books, lecture notes. It is simpler than LaTeX, more powerful than Word, as versatile as Markdown but, if I may say so, far more elegant than all of these.

1.1. Who this is for

This handbook is meant for:

- **Students and researchers** who want to produce flawless theses and articles without going crazy over formatting
- **Professionals** who want beautiful documents without depending on Word
- **The curious** who love understanding how things work
- **Anyone who has never programmed** – we really start from zero

No prior knowledge of programming, LaTeX, or HTML is required. You just need to know how to write.

1.2. How to use this handbook

The handbook is divided into seven parts, in increasing order of complexity:

Part I – Fundamentals (Ch. 1–5): What Typst is, how to install it, how to write simple text, bold, italic, headings, lists.

Part II – Additional elements (Ch. 6–10): Links, images, tables, math formulas, code blocks.

Part III – The Language (Ch. 11–16): The scripting system: variables, types, conditions, loops, functions.

Part IV – Styling (Ch. 17–19): The `set` and `show` rules. How to customize every aspect of the document.

Part V – A Bit of Advanced Layout (Ch. 20–23): Colors, boxes, professional layout, grids and columns.

Part VI – Project Organization (Ch. 24–25): Multiple files, modules, community packages.

Part VII — A Section of Real Projects (Ch. 26–28): Complete templates: academic document, CV, presentation.

Tip

Open typst.app in your browser while you read. Every time you see a code example, copy it and paste it into the editor: you will see the result in real time. If you learn by doing, hands-on, you learn faster and you also get quicker.

1.3. My typographic conventions

Throughout the text you will find these recurring elements:

code	Typst code, function names, files
Note	Useful information to keep in mind
Warning	Common mistakes to avoid
Tip	Tricks and best practices
Try it!	Hands-on exercise to run ¹

¹In the end I didn't use them... but I still liked the idea of having this tag, so here it is.

2. What Typst is and why use it

Typst is a *markup* and *programming* language designed to create documents of high typographic quality. In simple words: you write text in a file, add a few special instructions, and Typst automatically produces a professional PDF.

Think of Typst as something halfway between Microsoft Word and LaTeX.

2.1. Markup: what it is and how it works

First of all, let's clarify the term **markup**. A markup language is a system in which you write your content (text, images, equations...) together with some *markings*, special instructions that describe how that content should look.

The principle is to **separate content from form**.

- In Word, you click "Bold" and the text becomes bold.
- In Typst, as in LaTeX, you write `*bold text*` and Typst makes it bold when it compiles the PDF.

This approach has huge advantages:

- The document is a plain text file: lightweight and portable.
- Formatting is consistent throughout the document
- You can focus on the content while writing, without getting distracted by formatting menus
- You can automate anything: page numbers, tables of contents, cross-references, tables generated from code. You don't have to tear your hair out (if you have any... lucky you...) if you only want to change a piece of the table of contents or the order of the paragraphs — Typst does it all automatically.

2.2. Comparison with the most common tools

2.2.1. Typst vs Microsoft Word / LibreOffice

Aspect	Word / LibreOffice	Typst
Type	WYSIWYG (you see the result immediately)	Markup (you compile to see the result)
Learning curve	Low at first	Low — simpler than LaTeX
Consistency	Hard to maintain	Automatically perfect
Math formulas	Hard	Excellent, simple

File	Binary .docx format	Plain text .typ
Speed	Slow on large documents	Instant compilation
Automation	VBA macros (complex)	Integrated scripting (simple)
Cost	Expensive / free	Free and open-source

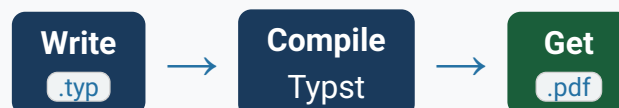
2.2.2. Typst vs LaTeX

LaTeX has been the undisputed king of academic typesetting for decades. Typst inherits its typographic excellence but solves its historical “problems”:

Aspect	LaTeX	Typst
Errors	Cryptic, programming-language-style messages	Very clear and precise messages
Compilation	Slow (seconds/minutes)	Instant and fast even on huge files
Syntax	Complicated and a bit dated	Concise and very new
Learning	Steep for a beginner	Very accessible
Packages	Thousands (CTAN)	Growing (Universe)
Typographic quality	Excellent	Excellent
Math	Great	Great, but more modern syntax

2.3. How Typst works: the workflow

The process is always the same:



In the online editor (typst.app), compilation happens automatically as you write: every time you pause your typing, the PDF updates in real time in the right-hand panel. It's always instant (it's instant even if you have a huge file, like this one; the important thing is to write/edit pieces of code and not paste everything in at once. In that case it might take a couple of seconds. (Don't vibe-code...).

2.4. A very brief historical note

Typst was born as the master's thesis project of two students at the Technical University of Berlin (TU Berlin), Laurenz Mädje and Martin Haug. The project opened its public beta in

March 2023, and since then it has gathered an enthusiastic community all over the world. It is open-source (Apache 2.0 license) and freely available both as a web app and as a command-line tool.

Note

In this handbook we use the online version of Typst (typst.app) for all examples. There is no need to install anything on your computer to get started. If you wish, you can install it on your device and write with whatever you like (I wrote this document in LazyVim).

3. Installation and first document

3.1. Option 1: Online editor (recommended to start)

The easiest way to get started is the official online editor. It requires no installation and works in any browser.

1. Open your browser and go to typst.app
2. Create a free account (or log in with Google/GitHub)
3. Click **“Empty document”** (or **“Start from template”** if you already want something ready-made)
4. The editor opens: the code on the left, the PDF preview on the right

The online editor has these features:

- Real-time PDF preview
- Syntax highlighting
- Autocompletion
- Instant error messages
- Ability to export the PDF (and more)
- Cloud storage of your projects
- Vim keybindings (recommended)²

Tip

If you're a beginner, I recommend — especially for the first chapters of this handbook — using the online editor exclusively. It's the quickest way to experiment without distractions, it's dead simple, and all you have to do is write.

²Speaking of the keyboard, it seems only fair to tell you that if you use the online version, as I recommend, there are tons of keyboard shortcuts. Discover and use them as much as you can, they speed up your work a lot!

3.2. Option 2: Local installation

For those who want to work offline or integrate Typst into their own workflow with a text editor like VS Code (or Vim!!).

3.2.1. Installation on Windows

```
winget install --id Typst.Typst
```

Or simply download the `.exe` file from the releases page: <https://github.com/typst/typst/releases>

3.2.2. Installation on macOS

With Homebrew:

```
brew install typst
```

3.2.3. Installation on Linux

With your system's package manager (♥ Arch ♥³, NixOS, etc.) or via:

```
cargo install --git https://github.com/typst/typst
```

Or download the pre-compiled binary from the releases page.

3.2.4. VS Code extension

Even though I don't like VS, I know many people use it, so if you too use Visual Studio Code (and are therefore a bad person), install the **Tinymist Typst** extension from the Extensions Marketplace. It provides:

- Integrated PDF preview
- Autocompletion and suggestions
- Document navigation
- Error highlighting

3.2.5. Compiling from the command line

Once installed, compile a file with:

```
typst compile my-document.typ
```

For watch mode (recompiles automatically on every save):

³I love Arch Linux, you must love Arch Linux too. I didn't include them in this handbook, but if you want these beautiful emoji too, you'll find all the codes at this address: <https://typst.app/docs/reference/symbols/emoji/>

```
typst watch my-document.typ
```

3.3. Structure of a .typ file

A Typst file has the `.typ` extension and is a plain text file. There is no mandatory structure: you can start writing right away.

A typical file is made up of:

```
// Document settings (optional but recommended)
#set document(title: "My document", author: "Stefano Coelati Rama")
#set page(paper: "a4", margin: 2cm)
#set text(font: "Linux Libertine", size: 11pt)

// From here on: the document content
= Document title

Here I write my normal text.

== A section

More text...
```

Lines starting with `//` are **comments**: Typst ignores them completely. They are useful for leaving notes to yourself.

3.4. Your first document: “Hello, W...Typst!”

Open the online editor, delete everything that’s there (if anything), and write this:

► Example

= Hello, Typst!

This is my first document written in Typst.
It's very easy to get started!

== A section

I can have ***bold***, *_italic_*, and even ``code``.

- First item

- Second item
- Third item

Compile (in the online editor it happens automatically) and you'll see the PDF appear on the right.

Try it!

In the online editor:

1. Copy the code above
2. Change the title to your name
3. Add another section with some text
4. Add a list with your three favorite foods

Note

In this handbook the code examples are shown with the `typ` language highlighted. When you copy the code, copy **only** the content, not the three opening and closing backticks (three backticks: ``,`,``) which are just the markup of the code block in the handbook (it's mandatory, otherwise it errors out; you'll see these "tags" often in the sections too).

4. Text and basic formatting

This is the most practical chapter: here you learn to do everything you normally do in a text editor, but in Typst.

4.1. Writing plain text

To write text, just write. Nothing special. Typst treats everything that doesn't start with a special character as plain text.

This is plain text.
You can write whatever you want.
Numbers: 42, 3.14, -7.
Punctuation: commas, periods, exclamation marks!

4.1.1. Paragraphs

To create a new paragraph, leave **one blank line** between two blocks of text:

► Paragraphs

TYPST CODE

This is the first paragraph.
This line continues in the same paragraph.

This is a second paragraph, separated by
a blank line.

RESULT

This is the first paragraph. This line
continues in the same paragraph.

This is a second paragraph, separated
by a blank line.

Note

Going to a new line in the code without leaving a blank line does **not** create a new paragraph; the text continues on the same line in the PDF. A bit like in LaTeX. To start a new paragraph, you need the blank line.

4.1.2. Line break without a new paragraph

If you want to go to a new line without creating a new paragraph (like in a poem or an address), use the backslash `\` followed by a space:

► Line break

TYPST CODE

```
Via Roma, 42 \  
00100 Rome \  
Italy
```

RESULT

Via Roma, 42
00100 Rome
Italy

Alternatively you can use the `#linebreak()` function.

4.2. Bold, italic, and underline

4.2.1. Bold

Wrap the text in asterisks `*...*`:

► Bold

TYPST CODE

RESULT

This word is **in bold**.
Several words together work too.

This word is **in bold**. **Several words together** work too.

4.2.2. Italic

Wrap the text in underscores `_..._`:

► Italic

TYPST CODE

This word is `_in italic_`.
`_A whole phrase_` can be in italic too.

RESULT

This word is *in italic*. A whole phrase can be in italic too.

4.2.3. Combining bold and italic

You can combine them by nesting one inside the other:

► Bold + Italic

TYPST CODE

This is `*_bold and italic_*` together.
Or `_text with part in *bold*_`.

RESULT

This is ***bold and italic*** together. Or text with part in **bold**.

4.2.4. Strikethrough text

Wrap the text in `#strike[text]` (no idea what you'd need it for, but there you go):

► Strikethrough text

TYPST CODE

The price was `#strike[100 euros]`, now it's 50 euros.

RESULT

The price was ~~100 euros~~, now it's 50 euros.

4.2.5. Inline code

Wrap it in backticks (yes, the backward apostrophe is called a backtick) ``...``:

► Inline code

TYPST CODE

Use the ``print()`` function to print.
The file is called ``document.typ``.

RESULT

Use the `print()` function to print. The
file is called `document.typ`.

4.2.6. Superscript and subscript

For superscript use `#super[...]`, for subscript `#sub[...]`:

► Superscript and subscript

TYPST CODE

Water is H`#sub[2]`O.
The formula is $E = mc$ `#super[2]`.
 x `#sub[1]` + x `#sub[2]` = x `#sub[1+2]`

RESULT

Water is H₂O. The formula is $E = mc^2$.
 $x_1 + x_2 = x_{1+2}$

Note

For real math formulas (with Greek letters, fractions, integrals...) there are the math modes `$...$` which we'll see in Chapter 9. Superscript and subscript with `#super` and `#sub` are only for plain text — use them sparingly.

4.3. Special characters and how to use them

Some characters have a special meaning in Typst. If you want to insert them as plain text, you have to “escape” them (do the escape, treat them as plain text) with the backslash `\`:

Character	How to type it	Special meaning
<code>*</code>	<code>*</code>	Bold
<code>_</code>	<code>_</code>	Italic
<code>~</code>	<code>\~</code>	Non-breaking space
<code>`</code>	<code>\`</code>	Inline code
<code>\$</code>	<code>\\$</code>	Math
<code>#</code>	<code>\#</code>	Code instruction
<code>@</code>	<code>\@</code>	Reference
<code><</code>	<code>\<</code>	Label
<code>\</code>	<code>\\</code>	Literal backslash

4.4. Typographic quotes

Typst automatically converts straight quotes into appropriate typographic (curly) quotes for the document's language:

► Typographic quotes

TYPST CODE

"Hello" become "curly quotes".
'Even' the 'single' ones work the same.

RESULT

"Hello" become "curly quotes". 'Even'
the 'single' ones work the same.

Note

Quotes adapt to the language set with `#set text(lang: "en")`. With English, the double quotes `"..."` become "curly quotes" and the single ones `'...'` become 'curly single quotes'. With `lang: "it"` (Italian), double quotes become «angular guillemets» and single quotes become "curly double quotes" (not <angular singles>).

4.5. Special spaces

Sometimes you need particular kinds of spacing:

Type	Syntax	Use
Normal space	<code>just type a space</code> (spacebar)	Between words
Non-breaking space	<code>~</code>	10 kg, Dr. Smith
Thin space	<code>#h(0.2em)</code>	After dots in abbreviations
Horizontal space	<code>#h(1cm)</code>	Custom spacing
Vertical space	<code>#v(1cm)</code>	Vertical spacing
Infinite space	<code>#h(1fr)</code>	Pushes text to the edges

► Special spaces

TYPST CODE

Dr.~Smith weighs 80~kg.
Left `#h(1fr)` Right

RESULT

Dr. Smith weighs 80 kg.
Left Right

4.6. Horizontal line

To insert a horizontal separator line, use the `#line()` function:

► Horizontal line

TYPST CODE

Text above the line.

```
#line(length: 100%, stroke: 0.5pt)
```

Text below the line.

RESULT

Text above the line.

Text below the line.

Warning

Unlike Markdown, in Typst three dashes `---` do **not** produce a horizontal line: they produce a long dash (*em dash*, “—”). Two dashes `--` give a medium dash instead (*en dash*, “-”). For a real separator line always use `#line(length: 100%)`.

5. Headings and document structure

Headings give structure to the document. In Typst you create them with the `=` (equals) symbol at the start of a line.

5.1. Heading levels

Typst supports six heading levels, created with an increasing number of `=`:

► Heading levels

= Chapter (level 1)

== Section (level 2)

=== Subsection (level 3)

==== Paragraph (level 4)

===== Level 5

===== Level 6

Warning

The `=` symbol must be at the **start of a line** and followed by a space. `=Title` (without a space) doesn't work as a heading, but is printed as plain text (= hi, see?).

5.2. Automatic numbering

Typst can number headings automatically. Just add to the settings:

► Heading numbering

```
#set heading(numbering: "1.1.1.")
= First chapter
== First section
=== First subsection
== Second section
= Second chapter
```

There are various numbering styles:

Numbering string	Description	Example
"1."	Arabic, one level	1. 2. 3.
"1.1."	Arabic, two levels	1.1. 1.2. 2.1.
"1.1.1."	Arabic, three levels	1.1.1.
"I."	Uppercase Roman	I. II. III.
"i."	Lowercase Roman	i. ii. iii.
"A."	Uppercase letters	A. B. C.
"a."	Lowercase letters	a. b. c.
"(1)"	Arabic in parentheses	(1) (2) (3)

5.3. The automatic table of contents with `#outline()`

One of Typst's powers is automatically generating the document's table of contents. Put this line where you want the TOC to appear:

```
#outline()
```

The TOC includes all headings and their pages. It's of course customizable:

```
// TOC up to level 3
#outline(depth: 3)
// TOC with a custom title
#outline(title: "Summary")
// TOC with automatic indentation
#outline(depth: 3, indent: 2em)
// TOC with only a certain element type
#outline(title: "List of figures", target: figure.where(kind: image))
#outline(title: "List of tables", target: figure.where(kind: table))
```

5.4. Headings outside the TOC

Sometimes you want a visual heading that doesn't appear in the TOC (for example in the preface or the appendix):

```
// Heading not included in the TOC and not numbered
#heading(level: 1, numbering: none, outlined: false)[Preface]
```

5.5. Labels and cross-references

You can assign a *label* to any heading (or figure, or equation) and then **refer** to it from another part of the document.

5.5.1. Assigning a label

After the heading, write `<label-name>` in angle brackets:

```
= Introductory Chapter <intro-chap>
== The history of Typst <history>
See the history section (@history) in chapter @intro-chap.
```

5.5.2. Referencing

Use `@label-name` to refer to the referenced item. Typst generates the link text automatically with the correct number:

- If it points to a chapter heading: "Chapter 1"
- If it points to a figure: "Figure 3"
- If it points to an equation: "Equation (5)"

► Reference to a section

```
== The Pythagorean Theorem <pythagorean-theorem>
As shown in @pythagorean-theorem, the square
of the hypotenuse equals the sum of the squares
of the legs.
```

Note

As in normal programming, reference labels must be **unique** in the document. You can't have two `<intro>` in the same file. So it's good practice to use descriptive names like `<chap-results>` instead of `<1>` (and if you can, comment! At least whoever reads the document in the future and wants to edit it won't go crazy).

5.6. Page break

To force a new chapter to start on a new page:

```
#pagebreak()
= New chapter
```

With `weak: true`, the page breaks only if it's not already at the start of a page:

```
#pagebreak(weak: true)
```

6. Lists

Lists are essential in almost every document. Typst supports three main kinds.

6.1. Bulleted list

Start each item with a dash `-` followed by a space:

► Bulleted list

TYPST CODE

RESULT

- First item

- First item
- Second item
- Third item

- Second item
- Third item

6.1.1. Custom markers

The default marker is `•`, but you can change it with `set list`:

```
#set list(marker: "→")  
- First item  
- Second item
```

Or you can specify different markers for each level:

```
#set list(marker: ([•], [-], [○]))
```

6.2. Numbered list

Start each item with `+` followed by a space:

► Numbered list

TYPST CODE

```
+ First point  
+ Second point  
+ Third point
```

RESULT

1. First point
2. Second point
3. Third point

6.2.1. Numbering style

```
// Numbering with letters  
#set enum(numbering: "a")  
+ First item // a)  
+ Second item // b)  
// Numbering with Roman numerals  
#set enum(numbering: "I.")
```

6.2.2. Starting from a specific number

```
#enum(start: 5,  
  [Fifth item],  
  [Sixth item],  
  [Seventh item],  
)
```

6.3. Nested lists

To create a sublist, indent the items with two spaces (or a tab):

► Nested list

TYPST CODE

- Fruit
 - Apples
 - Pears
 - Bananas
- Vegetables
 - Carrots
 - Spinach
- Grains

RESULT

- Fruit
 - Apples
 - Pears
 - Bananas
- Vegetables
 - Carrots
 - Spinach
- Grains

You can mix bulleted and numbered lists:

► Mixing lists

TYPST CODE

- + Prepare the ingredients:
 - Flour 500g
 - Water 300ml
 - Salt 10g
- + Knead
- + Let it rest:
 - At least 2 hours
 - In a cool place
- + Bake

RESULT

1. Prepare the ingredients:
 - Flour 500g
 - Water 300ml
 - Salt 10g
2. Knead
3. Let it rest:
 - At least 2 hours
 - In a cool place
4. Bake

6.4. Definition list (term: definition)

To create a glossary or list of terms with definitions, use `/ term: definition`:

► Definition list

TYPST CODE

```
/ Markup: A language that uses
special markers
in the text to describe its structure
and
formatting.
/ Compiler: A program that transforms
the source
code into an output file (in Typst's
case,
a PDF).
/ PDF: Portable Document Format. A
universally
readable file format.
```

RESULT

Markup A language that uses special markers in the text to describe its structure and formatting.

Compiler A program that transforms the source code into an output file (in Typst's case, a PDF).

PDF Portable Document Format. A universally readable file format.

6.5. Multi-paragraph items

List items can contain multiple paragraphs. Just indent the following text to the same level:

► Multi-paragraph items

TYPST CODE

```
- *First important item*
  This is the explanatory paragraph for
  the
  first item. It can be as long as you
  want.
  You can even have multiple
  paragraphs for the same item.
- *Second item*
  Another explanation.
```

RESULT

- **First important item** This is the explanatory paragraph for the first item. It can be as long as you want. You can even have multiple paragraphs for the same item.
- **Second item** Another explanation.

6.6. Spacing between items

By default list items are compact. You can increase the spacing between items:

```
#set list(spacing: 1.5em)
```

- First item
- Second item (now there's more space between items)

7. Links and references

7.1. Links to web pages

7.1.1. Basic syntax

```
#link("https://typst.app")[Visit Typst]
```

The first argument (in round brackets) is the URL, the second (in square brackets) is the text to display:

► Links to URLs

TYPST CODE

```
Visit #link("https://typst.app")[the  
official Typst website].  
Or: documentation at  
#link("https://typst.app/docs")  
[typst.app/docs].
```

RESULT

Visit [the official Typst website](https://typst.app). Or:
documentation at typst.app/docs.

7.1.2. Automatic URLs

If you write a URL directly in the text, Typst recognizes it and makes it clickable automatically:

Go to <https://typst.app> to get started.

7.1.3. Link without visible text (URL only)

```
#link("https://typst.app")  
// Shows the URL itself as text
```

7.1.4. Links to email addresses

Write to me at `#link("mailto:info@example.com")``[info@example.com]`

7.2. Footnotes

Footnotes are inserted with `#footnote[...]`:

► Footnotes

TYPST CODE

```
The term "markup"#footnote[  
  From "mark up", meaning  
  "to annotate" or "to mark".  
] dates back to the 1960s.
```

RESULT

The term “markup”⁴ dates back to the 1960s.

Footnotes are numbered automatically and appear at the bottom of the current page.

7.3. Links and references summary

Syntax	Effect
<code>#link("url")</code> <code>[text]</code>	External link with text
<code>https://...</code>	Automatic clickable URL
<code><label></code>	Assigns a label
<code>@label</code>	Reference to the label
<code>#footnote</code> <code>[text]</code>	Footnote

⁴From “mark up”, meaning “to annotate” or “to mark”.

8. Images and figures

8.1. Inserting an image

The function for inserting images is `#image("path")`. The path is relative to the location of the `.typ` file:

```
// Image in the same folder as the .typ file
#image("photo.jpg")
// Image in a subfolder
#image("images/chart.png")
// Image with fixed width
#image("logo.png", width: 5cm)
// Image with percentage width
#image("diagram.png", width: 80%)
// Image with fixed height
#image("photo.jpg", height: 4cm)
```

8.1.1. Supported formats

Typst supports the most common image formats:

- PNG (`.png`)
- JPEG/JPG (`.jpg` , `.jpeg`)
- SVG (`.svg`) — vector, ideal for logos and diagrams
- GIF (`.gif`) — only the first frame

8.1.2. The `fit` parameter

When you specify both `width` and `height`, you can control how the image is resized (so it doesn't make a mess when switching devices) with the `fit` parameter:

```
// "cover": fills the area (may crop)
#image("photo.jpg", width: 10cm, height: 6cm, fit: "cover")
// "contain": the whole image (with empty space on the sides)
#image("photo.jpg", width: 10cm, height: 6cm, fit: "contain")
// "stretch": distorts the image to fill
#image("photo.jpg", width: 10cm, height: 6cm, fit: "stretch")
```

8.2. The `#figure()` function: figures with a caption

For professional figures (with caption, automatic numbering, and inclusion in the list of figures) use `#figure()`:

```
#figure(
  image("chart.png", width: 80%),
  caption: [Temperature trend in 2024.],
) <fig-temperature>
```

8.2.1. Components of `#figure()`

The `figure` function accepts:

- The **content** as the first argument (the image, the table, etc.)
- `caption`: the caption (in square brackets)
- `label`: you can also put the label `<name>` after `)`

8.2.2. Customizing the supplement

By default figures show "Figure 1", "Figure 2", etc. You can customize it:

```
#set figure(supplement: "Fig.")
// Now it shows "Fig. 1", "Fig. 2"
// In English, Typst already uses "Figure"
// if you set lang: "en"
```

8.2.3. Figure placement

By default, figures are placed "inline" with the text. You can force the placement:

```
#figure(
  image("diagram.png"),
  caption: [System diagram.],
  placement: top, // Put it at the top of the page
)
// Other options: bottom, none (inline)
```

8.3. Alignment

You can center or align content with `#align()`:

```
// Centered
#align(center)[
  #image("logo.png", width: 4cm)
]
```

```
// To the right
#align(right)[
  #image("signature.png", width: 3cm)
]
```

8.4. Side-by-side images

To place two images side by side, use `#grid()`:

```
#grid(
  columns: (1fr, 1fr),
  gutter: 1em,
  figure(image("first.png"), caption: [First image.]),
  figure(image("second.png"), caption: [Second image.]),
)
```

8.5. Image formats and optimization

8.5.1. SVG images for vector graphics

The SVG format is preferable for logos, schemes, diagrams, and any graphic that must stay crisp at any scale. Typst includes the SVG renderer natively, no extra package is needed:

```
// SVG: crisp at any resolution
#image("diagram.svg", width: 100%)

// You can draw SVG inline in the document
#image(bytes("<svg xmlns='http://www.w3.org/2000/svg'
width='100' height='100'>
<circle cx='50' cy='50' r='40'
stroke='#1a3a5c' stroke-width='3' fill='#aed6f1'/>
</svg>"))
```

8.5.2. Images: local files only (no URLs)

Warning

Unlike other tools, Typst does **not** download images from the internet: `#image()` only accepts paths to local files (or in-memory data). Passing a URL produces the `file not found` error, because Typst interprets it as a path on disk.

If you want to use an image taken from the web, download it first and then reference it locally:

```
// Wrong: Typst does NOT download from the network → "file not found" error
// #image("https://example.com/logo.svg")

// Correct: download the file into the project folder and use the path
#image("images/logo.svg", width: 4cm)
```

8.5.3. Alternative text (accessibility)

For PDF/UA-1 compliance it's good practice to add a text description to the image:

```
#image("data-chart.png",
  width: 80%,
  alt: "Bar chart showing the monthly sales of 2024",
)
```

8.5.4. Clip and zoom: showing a portion of an image

```
// Show only the central part of the image
#box(
  clip: true,
  width: 5cm,
  height: 4cm,
)[
  #move(dx: -2cm, dy: -1cm)[
    #image("wide-photo.jpg", width: 9cm)
  ]
]
```

8.5.5. Side-by-side images with individual captions

```
#grid(
  columns: (1fr, 1fr),
  gutter: 1.5em,
  figure(
    image("first.png"),
    caption: [First experimental condition.],
  ) <fig-first>,
  figure(
    image("second.png"),
    caption: [Second experimental condition.],
  ) <fig-second>,
)
```

The results in @fig-first and @fig-second show...

8.5.6. Rotation and transformation

```
// Rotate the image
#rotate(90deg, image("diagram.png", width: 6cm))

// Scale (horizontal mirror)
#scale(x: -100%, image("logo.png", width: 3cm))

// Scale with percentage
#scale(80%, image("photo.jpg", width: 10cm))
```

8.5.7. Image as page background

```
#set page(
  background: place(
    center + horizon,
    image("background.png", width: 100%, height: 100%, fit: "cover"),
  ),
)
```

8.6. Practical tips

Tip

For scientific and academic documents, **always use** `#figure()` instead of `#image()` directly. This lets you:

- Have automatic figure numbering
- Reference with `@figure-name` in the text
- Generate the list of figures with `#outline(target: figure)`

Tip

Use **SVG** images when possible for logos, schemes, and charts: they stay crisp at any size in the PDF. For photographs use JPEG or PNG (mandatory if you want a transparent background).

Note

In the `typst.app` online editor you can upload images directly by dragging them onto the editor or using the upload button. Uploaded files are accessible from your project. Keeping your project tidy with folders matters for this too.

9. Tables

Tables in Typst are powerful and flexible. The main function is `#table()`.

9.1. Your first table

```
#table(  
  columns: 3, // number of columns  
  [Name], [Age], [City],  
  [Mario], [25], [Rome],  
  [Lucia], [30], [Milan],  
  [Carlo], [22], [Naples],  
)
```

Each element in `[...]` is a cell. Cells are filled row by row, from left to right.

► Basic table

TYPST CODE

```
#table(  
  columns: 3,  
  [Name], [Age], [City],  
  [Mario], [25], [Rome],  
  [Lucia], [30], [Milan],  
  [Carlo], [22], [Naples],  
)
```

RESULT

Name	Age	City
Mario	25	Rome
Lucia	30	Milan
Carlo	22	Naples

9.2. Defining columns

The `columns` parameter accepts:

- An integer (equal-width columns)
- A list of specific sizes

```
// 3 equal columns  
columns: 3  
// Specific widths  
columns: (3cm, 2cm, 4cm)  
// Proportions: 1fr = "the rest of the space"  
columns: (1fr, 2fr, 1fr)
```

```
// First and third columns equal, second twice as wide
// Mix of fixed and proportional
columns: (auto, 1fr, 2cm)
// First column: fits the content
// Second: takes the remaining space
// Third: fixed 2cm
```

9.3. Table headers

To create styled headers, use `table.header()`:

► Table with colored header

TYPST CODE

```
#table(
  columns: (1fr, auto, auto),
  stroke: 0.6pt + luma(180),
  fill: (col, row) => if row == 0
  { rgb("#1a3a5c") }
  else if calc.odd(row) { white }
  else { rgb("#f4f6f7") },
  table.header(
    [*Product*], [*Price*], [*Available*],
  ),
  [Laptop], [€999], [Yes],
  [Mouse], [€25], [Yes],
  [Monitor], [€350], [No],
  [Keyboard], [€45], [Yes],
)
```

RESULT

Product	Price	Available
Laptop	€999	Yes
Mouse	€25	Yes
Monitor	€350	No
Keyboard	€45	Yes

9.4. Content alignment

```
#table(
  columns: 3,
  align: (left, center, right),
  [Left-aligned text], [Centered text], [Right-aligned text],
  [Mario], [25], [Rome],
)
```

Or a global alignment:

```
#table(
  columns: 3,
  align: center, // All cells centered
  ...
)
```

Vertical alignment:

```
align: (left + top, center + horizon, right + bottom)
```

9.5. Alternating colors (zebra stripes)

The `fill` parameter accepts a function that receives the row and column and returns a color:

```
#table(
  columns: 3,
  fill: (col, row) => if calc.odd(row) { luma(240) } else { white },
  [A], [B], [C],
  [1], [2], [3],
  [4], [5], [6],
)
```

9.6. Borders (stroke)

```
// Uniform border
stroke: 0.5pt + black
// No border
stroke: none
// Horizontal border only
stroke: (top: 0.5pt + black, bottom: 0.5pt + black)
// Custom border per cell
// (in the fill function you can return stroke)
```

9.7. Cells spanning multiple columns/rows

```
#table(
  columns: 3,
  // This cell spans 2 columns
  table.cell(colspan: 2)[Columns 1 and 2 merged],
  [Column 3],
  [Normal], [Normal], [Normal],
)
```



```
// This cell spans 2 rows
table.cell(rowspan: 2)[2 rows],
[Row 1], [Row 1],
[Row 2], [Row 2],
)
```

9.8. Tables as figures

To get automatic numbering and be able to reference a table, wrap it in `#figure()`:

```
#figure(
  table(
    columns: 3,
    [A], [B], [C],
    [1], [2], [3],
  ),
  caption: [Experimental data.],
) <tab-data>
As shown in @tab-data...
```

Note

`#figure(table(...), ...)` automatically identifies the content as a table (not an image), so it will be listed in the list of tables separately from figures.

10. Mathematics

One of Typst's strengths is its math syntax: cleaner than LaTeX, easy to learn, and with typographically excellent results.

10.1. Math mode

As we mentioned earlier, there are (essentially) two ways to write math in Typst:

10.1.1. Inline math

Inline math appears in the body of the text, surrounded by `$...$` (single dollar, no spaces):

► Inline math

TYPST CODE

Euler's formula is $e^{i\pi} + 1 = 0$, considered the most beautiful in mathematics.
The Pythagorean theorem: $a^2 + b^2 = c^2$.

RESULT

Euler's formula is $e^{i\pi} + 1 = 0$, considered the most beautiful in mathematics. The Pythagorean theorem: $a^2 + b^2 = c^2$.

10.1.2. Block (display) math

Block math appears on its own line, centered and with larger symbols. You get it with `$...$` on a separate line, or by adding a space after the opening `$`:

► Block math

TYPST CODE

Einstein's equation:
 $E = mc^2$
Or Euler's identity in large:
 $e^{i\pi} + 1 = 0$

RESULT

Einstein's equation:
$$E = mc^2$$

Or Euler's identity in large:
$$e^{i\pi} + 1 = 0$$

Note

The difference between inline and display is the space after `$`:

- `$formula$` = inline (in the text flow)
- `$ formula $` = display (on its own line, centered)

Tip

For the clarity of the text, when introducing a new equation/math function, it is always preferable to use display (block). Everything stays much tidier. My advice is: always use display — it takes up more space but it's more readable. Inline works well if you have little math or you're just referring to something else (that was in a display).

10.2. Basic operators

Operation	Typst syntax	Result
Addition	<code>a + b</code>	$a + b$
Subtraction	<code>a - b</code>	$a - b$

Multiplication	<code>a dot b</code>	$a \cdot b$
Cross product	<code>a times b</code>	$a \times b$
Division	<code>a / b</code>	$\frac{a}{b}$
Equals	<code>a = b</code>	$a = b$
Not equal	<code>a != b</code>	$a \neq b$
Less than	<code>a < b</code>	$a < b$
Greater than	<code>a > b</code>	$a > b$
Less or equal	<code>a <= b</code>	$a \leq b$
Greater or equal	<code>a >= b</code>	$a \geq b$
Approximately	<code>a approx b</code>	$a \approx b$
Proportional	<code>a prop b</code>	$a \propto b$

10.3. Powers and subscripts

```
// Simple power
 $x^2$  //  $x^2$ 
 $x^{(n+1)}$  //  $x$  to the  $(n+1)$  – you need the parentheses!
// Simple subscript
 $x_i$ 
 $x_{(i,j)}$  // with a compound subscript
// Together
 $x_i^2$ 
 $x_{(i+1)}^{(2n)}$  // with compound expressions
```

Powers and subscripts

TYPST CODE

$x^2 + y^2 = r^2$ is the circle.
 $a_n = a_{(n-1)} + a_{(n-2)}$ is Fibonacci.
The series $\sum_{(n=0)}^{(+infinity)} x^n$
= $1/(1-x)$
converges for $|x| < 1$.

RESULT

$x^2 + y^2 = r^2$ is the circle. $a_n = a_{n-1} + a_{n-2}$ is Fibonacci. The series $\sum_{n=0}^{+\infty} x^n = \frac{1}{1-x}$ converges for $|x| < 1$.

η	eta	θ	theta	λ	lambda
μ	mu	ν	nu	ξ	xi
π	pi	ρ	rho	σ	sigma
τ	tau	φ	phi	ψ	psi
ω	omega	Δ	Delta	Σ	Sigma
Ω	Omega	Λ	Lambda	Π	Pi

10.6.2. Other common symbols

Symbol	Code	Symbol	Code
∞	infinity	∂	partial
∇	nabla	$\forall$	forall
$\exists$	exists	$\in$	in
$\subset$	subset	$\cup$	union
$\cap$	inter	$\rightarrow$	arrow.r
$\leftrightarrow$	arrow.l.r	$\Rightarrow$	arrow.r.double
$\pm$	plus.minus	$\mp$	minus.plus

10.7. Sums, products, limits

```
// Sum
$sum_(i=0)^n a_i$
// Product
$product_(k=1)^n k = n!$
// Limit
$lim_(x -> 0) frac(sin x, x) = 1$
// In display mode (larger)
$ sum_(i=0)^n i^2 = frac(n(n+1)(2n+1), 6) $
```

► Sum and limit

TYPST CODE

```
$ sum_(i=1)^n i = frac(n(n+1), 2) $
$ lim_(n->infinity) (1 + 1/n)^n = e $
```

RESULT

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$
$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e$$

10.8. Vectors and matrices

```
// Column vector
$vec(a, b, c)$
// Matrix
$mat(a, b; c, d)$ // The semicolon separates rows
// Matrix with delimiters
$mat(delim: "[", a, b; c, d)$
$mat(delim: "|", a, b; c, d)$ // Determinant
```

► Matrices and vectors

TYPST CODE

The rotation matrix is:
 $R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$
 A linear system:
 $\text{mat}(a, b; c, d) \cdot \text{vec}(x, y) = \text{vec}(e, f)$

RESULT

The rotation matrix is:

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

A linear system:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} e \\ f \end{pmatrix}$$

10.9. Text inside formulas

To insert plain text inside a formula use quotes:

```
$f(x) = x^2 \text{ quad "for" } x > 0$
```

► Piecewise function

TYPST CODE

```
$f(x) = \text{cases}(x^2 \text{ &"if" } x \geq 0, -x^2 \text{ &"if" } x < 0)$
```

RESULT

$$f(x) = \begin{cases} x^2 & \text{if } x \geq 0 \\ -x^2 & \text{if } x < 0 \end{cases}$$

10.10. Numbered formulas

For numbered formulas in a scientific document:

```
#set math.equation(numbering: "(1)")
$ E = m c^2 $ <eq-einstein>
As shown in @eq-einstein...
```

10.11. Spaces in math

In math mode normal spaces are ignored. To add spacing use:

```
$a quad b$ // large space
$a wide b$ // very large space
$a thin b$ // thin space
$a med b$ // medium space
$a space b$ // normal space
```

11. Code blocks

Typst handles the display of source code excellently with syntax highlighting (**syntax highlighting**) for dozens of languages.

11.1. Inline code

Use a single backtick for inline code in the text:

```
Use the `print()` function to display output.
The variable `x` has value `42`.
```

11.2. Code block

Use three backticks for a code block, with the name of the language after the opening backticks:

```
```python
def factorial(n):
 if n <= 1:
 return 1
 return n * factorial(n - 1)
print(factorial(5)) # Output: 120
```
```

► Python code with highlighting

```
```python
def greet(name):
 return f"Hello, {name}!"

print(greet("Typst"))
```
```

11.3. Supported languages

Typst supports highlighting for very many languages:

| Language | Identifier | Language | Identifier |
|----------|-----------------------|------------|--|
| Python | <code>python</code> | JavaScript | <code>javascript</code> or <code>js</code> |
| Java | <code>java</code> | C/C++ | <code>c</code> , <code>cpp</code> |
| Rust | <code>rust</code> | Go | <code>go</code> |
| HTML | <code>html</code> | CSS | <code>css</code> |
| SQL | <code>sql</code> | Bash | <code>bash</code> , <code>sh</code> |
| LaTeX | <code>latex</code> | Typst | <code>typ</code> |
| JSON | <code>json</code> | YAML | <code>yaml</code> |
| Markdown | <code>markdown</code> | R | <code>r</code> |
| MATLAB | <code>matlab</code> | Julia | <code>julia</code> |

11.4. Line numbers

With the `codly` package (which we'll see in Chapter 25) you can add line numbers and many other options. For now, Typst's basic display doesn't include line numbers.

11.5. Custom code style

You can customize the appearance of all code using `#set raw`:

```
// Change the code font
#set raw(theme: "halcyon") // dark theme
// Or via a show rule (as done in this handbook)
#show raw.where(block: true): it => {
  block(
    fill: luma(245),
```



```
inset: 1em,  
radius: 5pt,  
)[#it]  
}
```

11.6. Line numbers and manual highlighting

You can simulate line numbers with a grid layout:

```
#let code-with-numbers(src) = {  
  let lines = src.split("\n")  
  let cells = ()  
  for (i, line) in lines.enumerate() {  
    cells.push(text(fill: luma(150), str(i+1)))  
    cells.push(raw(line))  
  }  
  grid(columns: (auto, 1fr), gutter: 0.5em, ..cells)  
}
```

12. Introduction to code: the `#` symbol

Up to now you've used Typst as a markup language: you write text, add `*bold*`, `= Headings`, and so on. But Typst has a second face: it's also a **programming** language. In this chapter we start exploring it.

12.1. Two modes: Markup and Code

In Typst there are two operating modes:

| | Markup mode | Code mode |
|------------------|--|---|
| How you enter it | It's the default | With the <code>#</code> symbol |
| What you write | Plain text with markings | Programming instructions |
| Examples | <code>*bold*</code> , <code>= Heading</code> | <code>#let x = 5</code> , <code>#if cond [...]</code> |
| Purpose | Document content | Logic, variables, functions |

The `#` symbol is the **hash** (pound sign). When you write it in the text, you're telling Typst: "what follows is not text, it's a code instruction".

12.2. The `#` symbol in action

Here's how switching between the two modes works:

```
This is plain text.  
#let name = "Fabiana" // Define a variable  
Hi, #name! How are you? You look great! // Use the variable in the text  
Calculation: #(2 + 3) makes five. // Inline calculation  
#if true [This sentence appears.] // Inline condition
```

Every line that starts with `#` (or that has `#` in the middle of the text) contains Typst code.

12.3. Inline expressions with `#`

You can insert the result of any expression into the text using `#(expression)` or `#variable`:

► Inline expressions

TYPST CODE

```
The result of 7 × 6 is #(7 * 6).\nThe current year is\n#datetime.today().year().\nThe text in uppercase: #("hello\nworld".split(" ").map(s =>\nupper(s)).join(" ")).
```

RESULT

The result of 7 × 6 is 42.
2026.
The text in uppercase: HELLO WORLD.

12.4. Code blocks with `{ }`

Curly braces `{ }` contain a **code block**: multiple instructions executed in sequence. The result is the value of the last expression.

```
#{\n  let a = 10\n  let b = 20\n  a + b // Returns 30\n}
```

The returned value appears in the document output:

► Code block

TYPST CODE

```
The maximum of 7 and 3 is #{
  let a = 7
  let b = 3
  if a > b { a } else { b }
}.
```

RESULT

The maximum of 7 and 3 is 7.

12.5. Content blocks with `[]`

Square brackets `[]` contain a **content block**: text and markup that is treated as document content. You can think of them as “mini-documents”:

```
// Content as a function argument
#block(fill: yellow)[
  This text is inside a yellow block.
  I can use *bold* and _italic_ normally.
]
// Content assigned to a variable
#let message = [Hi, *world*!]
#message // Prints the content
```

► Content blocks

TYPST CODE

```
#let notice = [
  *Warning:* this is an important
  notice.
]
#block(fill: rgb("#fff3cd"), inset: 0.8em,
  radius: 5pt)[#notice]
```

RESULT

Warning: this is an important notice.

12.6. The fundamental difference: `{ }` vs `[]`

This distinction is **crucial** in Typst and it's worth restating it with a table:

| | Braces <code>{ }</code> | Brackets <code>[]</code> |
|-----------------------|-------------------------|--------------------------|
| Contain | Code (instructions) | Content (text/markup) |
| Inside you use | Programming syntax | Markup syntax |

| | | |
|------------------------------|------------------------------|-----------------|
| Does *bold* work? | No | Yes |
| Does #let x = 5 work? | Yes | With # |
| They return | Value of the last expression | A content block |

12.7. The basic principle: **#** for all code

Every time you want to do something “programmatic” in the middle of markup, you use **#**:

```
// Call a function:
#upper("hi") // "HI"
// Define a variable:
#let pi = 3.14159
// Use a variable:
Pi is about #pi.
// Condition:
#if 2 > 1 [True!] else [False!]
// Loop:
#for i in (1, 2, 3) [Number #i. ]
// Built-in function:
#text(fill: red)[Red text]
// Setting:
#set text(size: 14pt)
```

Note

If you’re in the middle of a code block (**#{ ... }**), you don’t need the **#** before everything – you’re already in code mode. The **#** is only needed when you’re in markup mode and want to “step out” momentarily into code.

12.8. The execution context

Typst executes code in the order it appears in the document. Variables defined with **#let** remain available for the whole document (or for the scope in which they are defined):

► Variables in the document

TYPST CODE

```
#let title = "My Book: Short
Chronotropic Fragments"
#let author = "Stefano Coelati Rama"
#let year = 2026
```

RESULT

**My Book: Short Chronotropic
Fragments** Written by **Stefano
Coelati Rama** in 2026.

```
= #title  
Written by *#author* in #year.
```

13. Data types

As in normal programming, in Typst too every value has a **type**, a category that determines what you can do with it. Knowing the types is essential to understand why some code works and other code doesn't.

13.1. Overview of types

Typst has these main types:

| Type | Example | Description |
|----------|----------------|------------------------|
| string | "hello" | Text in quotes |
| int | 42 , -7 | Integer number |
| float | 3.14 , -0.5 | Decimal number |
| bool | true , false | True or false |
| length | 2cm , 12pt | Length measure |
| ratio | 50% | Percentage |
| color | rgb("#ff0000") | Color |
| array | (1, 2, 3) | Ordered list of values |
| dict | (a: 1, b: 2) | Key-value dictionary |
| content | [*Hello*] | Document content |
| none | none | Absence of value |
| auto | auto | Automatic value |
| function | x => x + 1 | A function |

13.2. Strings

A string is a sequence of characters enclosed in **double quotes** `"..."`:

```
#let name = "Mario"  
#let greeting = "Hello, world!"  
#let empty = "" // Empty string
```

```
// There is no single-quote string in Typst:  
// 'wrong' ← ERROR!  
// "correct" ← OK
```

13.2.1. String operations

```
// Concatenation  
#let full-name = "Mario" + " " + "Rossi"  
// Length  
#"hello".len() // 5  
// Upper/lowercase  
#upper("hello") // "HELLO"  
#lower("HELLO") // "hello"  
// Contains  
#"hello world".contains("world") // true  
// Replace  
#"hello world".replace("world", "Typst") // "hello Typst"  
// Split  
#"a,b,c".split(",") // ("a", "b", "c")  
// Trim spaces  
#" hello ".trim() // "hello"
```

13.2.2. String interpolation

To insert values inside a string, it's more convenient to use concatenation or markup directly:

```
#let name = "Mario"  
// In markup:  
Hi, #name!  
// In a string (with concatenation):  
#let msg = "Hi, " + name + "!"
```

13.3. Integers (int)

An integer is a number without a decimal part:

```
#let n = 42  
#let negative = -7  
#let zero = 0  
// Operations:  
#(10 + 3) // 13  
#(10 - 3) // 7  
#(10 * 3) // 30  
#(10 / 3) // 3.333... (becomes a float!)  
#(10 // 3) // 3 (integer division)
```

```
#(10 mod 3) // 1 (remainder of the division)
#calc.pow(2, 8) // 256 (power)
#calc.abs(-5) // 5 (absolute value)
```

13.4. Decimal numbers (float)

Floats have a decimal point:

```
#let pi = 3.14159
#let gravity = -9.81
// Predefined constant:
#calc.pi // 3.141592...
#calc.e // 2.718281...
// Math functions:
#calc.sqrt(16.0) // 4.0
#calc.sin(calc.pi / 6) // 0.5
#calc.cos(0.0) // 1.0
#calc.log(100.0) // 2.0 (log base 10)
#calc.ln(calc.e) // 1.0 (natural log)
#calc.floor(3.7) // 3 (rounds down)
#calc.ceil(3.2) // 4 (rounds up)
#calc.round(3.5) // 4 (rounds)
```

13.5. Booleans (bool)

A boolean is simply `true` or `false`:

```
#let is-logged-in = true
#let has-errors = false
// Logical operations:
#(true and false) // false
#(true or false) // true
#(not true) // false
// Comparisons (return bool):
#(5 > 3) // true
#(5 == 5) // true (DOUBLE equals for comparison!)
#(5 != 3) // true
#("a" == "a") // true
```

Warning

In Typst (as in most languages), to **compare** two values you use the **double equals** `==`, while to **assign** a value to a variable you use the single equals `=`. Confusing the two is a common mistake!

```
// Assignment (in let): make the variable x hold 5:  
#let x = 5  
// Comparison (in conditions): is x equal to 5? (returns the boolean yes/no):  
#if x == 5 [Yes!]
```

13.6. Lengths and measures

Lengths are essential for layout. Typst supports various units:

| Unit | Example | Description |
|------|---------|--|
| pt | 12pt | Typographic point (1/72 inch) |
| cm | 2.5cm | Centimeter |
| mm | 10mm | Millimeter |
| in | 1in | Inch (25.4mm) |
| em | 1.5em | Relative to the current font size |
| % | 80% | Percentage of the container |
| fr | 1fr | Fraction of available space (only in grid) |

```
// Operations on lengths:  
#(2cm + 5mm) // 2.5cm  
#(1in - 1cm) // ~1.546cm  
#(2 * 1.5cm) // 3cm
```

13.7. Colors

Colors can be specified in various ways:

```
// RGB with values 0-255  
#rgb(255, 0, 0) // Pure red  
#rgb(0, 128, 255) // Blue  
// RGB with a hex string  
#rgb("#ff0000") // Red  
#rgb("#2471a3") // Medium blue  
// Grayscale  
#luma(0) // Black  
#luma(128) // Medium gray  
#luma(255) // White  
// CMYK (for print)  
#cmyk(0%, 100%, 100%, 0%) // CMYK red  
// Predefined colors  
color.red
```



```
color.blue
color.green
color.black
color.white
color.gray
color.yellow
color.orange
color.purple
// Modify a color
#color.red.darken(30%) // Darker red
#color.blue.lighten(50%) // Lighter blue
#color.green.transparentize(70%) // Transparent green
```

13.8. Arrays (lists)

An array is an ordered sequence of values:

```
// Create an array
#let numbers = (1, 2, 3, 4, 5)
#let names = ("Mario", "Lucia", "Carlo")
#let mixed = (1, "two", true, 3.14)
// Access elements (indices start at 0!)
#numbers.at(0) // 1 (first element)
#numbers.at(4) // 5 (fifth element)
#numbers.first() // 1
#numbers.last() // 5
// Length
#numbers.len() // 5
// Add elements
#let numbers = (1, 2, 3, 4, 5)
#let extended = numbers + (6,) // (1, 2, 3, 4, 5, 6)
// push method
#numbers.push(6) // Adds 6 at the end: now numbers = (1, 2, 3, 4, 5, 6).
// push() modifies the variable in place. Arrays have copy semantics:
// passing one to a function does not touch the original.
// Slice (subarray)
#numbers.slice(1, 3) // (2, 3) — from the second to the third
// Map (transform each element)
#numbers.map(n => n * 2) // (2, 4, 6, 8, 10)
// Filter
#numbers.filter(n => n > 2) // (3, 4, 5)
// Sum/concatenation
#(1, 2) + (3, 4) // (1, 2, 3, 4)
// Contains?
#numbers.contains(3) // true
// Reverse
#numbers.rev() // (5, 4, 3, 2, 1)
```

```
// Sort
#(3, 1, 4, 1, 5).sorted() // (1, 1, 3, 4, 5)
```

Note

In Typst, like in all serious programming languages (right, R?), array indices **start at 0**: the first element is `.at(0)`, the second `.at(1)`, etc. This is standard in almost all modern programming languages.

13.9. Dictionaries (dict)

A dictionary maps **keys** (names) to **values**:

```
// Create a dictionary
#let person = (
  name: "Mario",
  age: 25,
  city: "Rome",
)
// Access values
#person.name // "Mario"
#person.at("age") // 25 (alternative syntax)
// Keys are strings (without quotes for convenience)
// Add or modify
#let person = (name: "Mario", age: 25, city: "Rome")
#let new_person = (..person, email: "mario@example.com")
// insert method: adds (or updates) a key, modifying the variable in place
#person.insert("email", "mario@example.com") // now person also contains email
// The keys
#person.keys() // ("name", "age", "city")
// The values
#person.values() // ("Mario", 25, "Rome")
// Contains a key?
#person.keys().contains("name") // true
```

13.10. Content

The `content` type is Typst's native type for document content. Every text, image, table... is content:

```
// Create content with brackets:
#let text-content = [Hi, *world*!]
// Content is different from a string:
#let s = "Hi" // string
```

```
#let c = [Hi] // content (may contain markup)
// Use content as an argument
#block(fill: yellow)[#text-content]
```

13.11. none and auto

```
// none: absence of value
#let value = none
#if value == none [There's nothing]
// auto: value computed automatically by Typst
#set page(header: auto) // Typst decides on its own
```

13.12. Discovering a value's type

The `type()` function tells you what type a value is:

► The `type()` function

TYPST CODE

```
The type of 42 is: #type(42)
The type of "hello" is: #type("hello")
The type of 3.14 is: #type(3.14)
The type of true is: #type(true)
The type of 2cm is: #type(2cm)
The type of (1,2,3) is: #type((1,2,3))
```

RESULT

The type of 42 is: `int` The type of "hello" is: `str` The type of 3.14 is: `float` The type of true is: `bool` The type of 2cm is: `length` The type of (1,2,3) is: `array`

13.13. Type conversions

```
#int(3.7) // 3 (truncates, doesn't round)
#float(5) // 5.0
#str(42) // "42"
#str(3.14) // "3.14"
#int("42") // 42
```

14. Variables

A variable is a **name** to which you associate a **value**. Once defined, you can use that name instead of rewriting the value every time.

14.1. Defining a variable with `#let`

The keyword for defining variables is `let`:

```
#let x = 5
#let name = "Mario"
#let active = true
#let main-color = rgb("#1a3a5c")
```

14.2. Using a variable

Once defined, you use the variable by writing `#variable-name`:

► Using variables

TYPST CODE

```
#let author = "Giulia Bianchi"
#let year = 2026
#let title = "My Thesis"
= #title
Author: *#author* --- Year: #year
```

RESULT

My Thesis Author: **Giulia Bianchi**
— Year: 2026

14.3. Why use variables?

Variables are useful for:

1. **Avoiding repetition:** if the author's name appears in 10 places, you just change it once in the definition.
2. **Making the code readable:** `accent-color` is clearer than `rgb("#1a3a5c")` repeated everywhere.
3. **Reusing content:** you can define a block of text and insert it multiple times.

► Variables for configuration

```
#let title-color = rgb("#1a3a5c")
#let text-color = luma(50)
#let spacing = 1.5em
#set text(fill: text-color)
#set par(leading: spacing)
#text(fill: title-color, size: 16pt, weight: "bold")[
  My nicely configured title
]
A paragraph with the spacing set above.
```

14.4. Variable scope

Variables are available from the point where they are defined and in everything that comes after:

```
#let a = 10
// 'a' is available here
#{
  let b = 20 // 'b' exists only inside these braces
  a + b // OK: a is defined outside
}
// 'b' is NOT available here (outside the scope)
```

Note

A variable defined outside any `{ }` or `[ ]` block is available for the whole document. One defined inside a `{ }` block exists only inside that block.

14.5. Variables that hold content

One of the most powerful things: variables can hold content blocks (markup) to reuse:

► Reusable content variable

TYPST CODE

```
#let safety-notice = [
  *Warning:* Before proceeding, make
  sure
  you've saved your work.
```

RESULT

Step 1 Warning: Before proceeding, make sure you've saved your work. Here I describe the first step...

```
1
== Step 1
#safety-notice
Here I describe the first step...
== Step 2
#safety-notice
Here I describe the second step...
```

Step 2 Warning: Before proceeding, make sure you've saved your work. Here I describe the second step...

14.6. Constants vs variables

In Typst there's no formal distinction between "constant" and "variable". By convention, values that don't change are defined at the start of the document and used with descriptive names.

14.7. Operations and calculations

Numeric variables support all arithmetic operations:

► Calculations with variables

TYPST CODE

```
#let width = 10cm
#let height = 7cm
#let area = width * height / (1cm *
1cm)
Width: #width
Height: #height
Area: #area cm2
```

RESULT

Width: 10cm Height: 7cm Area: 70
cm²

15. Conditions: if, else if, else

Conditions let you show or run things **only when a certain condition is true**. They are a fundamental tool in programming.

15.1. The basic if structure

```
#if condition {  
  // Code run if the condition is true  
}
```

Or with content:

```
#if condition [  
  Content shown if the condition is true.  
]
```

► Basic if

TYPST CODE

```
#let temperature = 35  
#if temperature > 30 [  
  It's very hot today!  
]  
#if temperature < 0 [  
  Warning: below-zero temperature!  
]
```

RESULT

It's very hot today!

15.2. if ... else

To handle the case where the condition is false:

```
#if condition [  
  Shown if true.  
] else [  
  Shown if false.  
]
```

► if ... else

TYPST CODE

```
#let score = 72  
#if score >= 60 [  
  *Passed!* Score: #score/100.  
]
```

RESULT

Passed! Score: 72/100.

```
] else [  
  *Failed.* Score: #score/100.  
]
```

15.3. if ... else if ... else

For multiple cases:

► Multiple conditions

TYPST CODE

```
#let grade = 7  
#if grade >= 9 [  
  Excellent!  
] else if grade >= 7 [  
  Good.  
] else if grade >= 6 [  
  Sufficient.  
] else [  
  Insufficient.  
]
```

RESULT

Good.

15.4. Comparison operators

| Operator | Meaning | Example |
|--------------------|------------------|--------------------------|
| <code>==</code> | Equal | <code>x == 5</code> |
| <code>!=</code> | Not equal | <code>x != 5</code> |
| <code><</code> | Less than | <code>x < 10</code> |
| <code>></code> | Greater than | <code>x > 0</code> |
| <code><=</code> | Less or equal | <code>x <= 100</code> |
| <code>>=</code> | Greater or equal | <code>x >= 18</code> |

15.5. Logical operators

You can combine multiple conditions:

| Operator | Meaning | Example |
|----------|-------------------|-------------------------------------|
| and | Both true | <code>x > 0 and x < 10</code> |
| or | At least one true | <code>x < 0 or x > 100</code> |
| not | Negation | <code>not (x == 0)</code> |

► Combined conditions

TYPST CODE

```
#let age = 25
#let has-license = true
#if age >= 18 and has-license [
  Can drive.
] else if age >= 18 and not has-license [
  Is an adult but has no license.
] else [
  Is a minor.
]
```

RESULT

Can drive.

15.6. if as an expression

In Typst, `if` returns a value! This means you can use it where you would normally put a value:

► if as an expression

TYPST CODE

```
#let dark-mode = true
#let background = if dark-mode
{ rgb("#1a1a2e") } else { white }
#let text-color = if dark-mode { white }
else { black }
#block(fill: background, inset: 1em,
radius: 5pt)[
  #text(fill: text-color)[
    This is a box in #if dark-mode [dark]
    else [light] mode.
  ]
]
```

RESULT

This is a box in dark else [light] mode.

16. Loops: for and while

Loops let you **repeat** an operation multiple times. They are fundamental for generating content automatically.

16.1. The for loop

16.1.1. Iterating over an array

The most common form: run something **for each element** of a collection:

► for over an array

TYPST CODE

```
#let fruit = ("Apples", "Pears",  
"Bananas", "Oranges")  
#for f in fruit [  
  - #f  
]
```

RESULT

- Apples
- Pears
- Bananas
- Oranges

16.1.2. Iterating with range

`range(n)` generates the numbers from 0 to n-1:

► for with range

TYPST CODE

```
#for i in range(5) [  
  The number #i squared is #(i * i).  
]
```

RESULT

The number 0 squared is 0.
The number 1 squared is 1.
The number 2 squared is 4.
The number 3 squared is 9.
The number 4 squared is 16.

`range(start, end)` generates from `start` to `end - 1`:

► Times table of 7

TYPST CODE

```
#for n in range(1, 11) [  
  #n × 7 = #(n * 7) \  
]
```

RESULT

```
1 × 7 = 7  
2 × 7 = 14  
3 × 7 = 21  
4 × 7 = 28  
5 × 7 = 35  
6 × 7 = 42  
7 × 7 = 49  
8 × 7 = 56  
9 × 7 = 63  
10 × 7 = 70
```

16.1.3. range with step

```
// range(start, end, step: step)  
#for n in range(0, 20, step: 5) [#n,]  
// 0, 5, 10, 15,
```

16.1.4. Generating a table with for

Here's a practical example: generating a multiplication table:

► Table with a for loop

```
#let n-max = 5  
#table(  
  columns: n-max + 1,  
  fill: (col, row) => if col == 0 or row == 0 { C.blu }  
  else if calc.odd(col + row) { white }  
  else { rgb("#f4f6f7") },  
  // First row: header  
  table.cell(fill: C.blu)[],  
  ..range(1, n-max + 1).map(n =>  
    table.cell()[#text(fill: white, weight: "bold")[#n]]  
  ),  
  // Data rows  
  ..range(1, n-max + 1).map(i =>  
    (  
      table.cell()[#text(fill: white, weight: "bold")[#i]],  
      ..range(1, n-max + 1).map(j => [(i * j)]),  
    )  
  )  
)
```

```
    ).flatten(),  
  )  
}
```

16.1.5. Iterating with an index

To get both the index and the value, use `.enumerate()`:

► enumerate()

TYPST CODE

```
#let colors = ("Red", "Green", "Blue")  
#for (i, color) in colors.enumerate() [  
  #(i + 1). #color  
]
```

RESULT

1. Red
2. Green
3. Blue

16.1.6. for returns an array

The `for` loop collects the results into an array. If you don't want it to return anything (e.g. you're only using side effects), you can ignore it:

```
// The for returns an array of results:  
#let squares = for i in range(1, 6) { i * i }  
// squares = (1, 4, 9, 16, 25)  
// To convert to content, use map + join:  
#range(1, 6).map(i => str(i * i)).join(", ")  
// Result: 1, 4, 9, 16, 25
```

16.2. The while loop

The `while` continues as long as the condition is true:

```
#let i = 0  
#while i < 5 {  
  i += 1  
  str(i) + " "  
}  
// Result: 1 2 3 4 5
```

Warning

With the `while` loop, be **very** careful not to create an infinite loop (a condition that stays always true). Typst could hang. Always make sure the condition eventually becomes `false`.

► Powers of 2 with while**TYPST CODE**

```
#let n = 1
#while n <= 1000 {
  [#n]
  n *= 2
}
```

RESULT

1 2 4 8 16 32 64 128 256 512

16.3. break and continue

Inside a loop you can:

- `break`: exit the loop immediately
- `continue`: skip to the next iteration

```
// Print the numbers from 1 to 10, but skip 5 and stop at 8
#for i in range(1, 11) {
  if i == 5 { continue } // Skip 5
  if i == 9 { break } // Stop before 9
  [#i]
}
// Result: 1 2 3 4 6 7 8
```

16.4. Generating complex content with loops

Practical example: generating a series of info cards:

► Cards generated with a loop**TYPST CODE**

```
#let languages = (
  (name: "Python", year: 1991, use:
```

RESULT

Python (1991) — Data and AI

```
"Data and AI"),  
  (name: "Rust", year: 2015, use:  
    "Systems"),  
  (name: "Typst", year: 2023, use:  
    "Documents"),  
)  
#for l in languages {  
  block(  
    fill: rgb("#eaf4fb"),  
    inset: 0.8em,  
    radius: 5pt,  
    stroke: (left: 3pt + rgb("#2471a3")),  
    below: 0.7em,  
  ) [  
    *#l.name* (#l.year) – #l.use  
  ]  
}
```

Rust (2015) – Systems

Typst (2023) – Documents

17. Functions

A function is a block of code that you give a name and that you can **call** whenever you need it, passing it different inputs to get different outputs. Think of a function as a recipe: you give it the ingredients (parameters) and it returns the dish (result).

17.1. Typst's built-in functions

Typst has hundreds of ready-made functions. You've already used many:

```
#text(size: 14pt, fill: red)[Hello] // 'text' function  
#image("photo.png", width: 80%) // 'image' function  
#table(columns: 3, ...) // 'table' function  
#upper("hello") // "HELLO"  
#lower("HELLO") // "hello"  
#calc.sqrt(16) // 4.0
```

17.2. Defining your own functions

Use `let` to define a custom function:

```
#let function-name(param1, param2) = body
```

17.2.1. Simple function

► First function

TYPST CODE

```
// A function that greets  
#let greet(name) = "Hello, " + name +  
"!"  
#greet("Mario")  
#greet("Lucia")
```

RESULT

Hello, Mario!
Hello, Lucia!

17.2.2. Function with a complex body

For functions with multiple instructions, use braces:

► Function with logic

TYPST CODE

```
#let classify-grade(v) = {  
  if v >= 9 { "Excellent" }  
  else if v >= 7 { "Good" }  
  else if v >= 6 { "Sufficient" }  
  else { "Insufficient" }  
}  
- Grade 10: #classify-grade(10)  
- Grade 7.5: #classify-grade(7.5)  
- Grade 5: #classify-grade(5)
```

RESULT

- Grade 10: Excellent
- Grade 7.5: Good
- Grade 5: Insufficient

17.3. Named parameters and default values

You can give parameters a default value. This makes them **optional** — if you don't pass them, the default is used:

► Parameters with defaults

TYPST CODE

RESULT

```
#let badge(label, color:
  rgb("#2471a3"), text-color: white) = {
  box(
    fill: color,
    inset: (x: 0.7em, y: 0.3em),
    radius: 4pt,
  )[
    #set text(fill: text-color, size: 9pt,
    weight: "bold")
    #label
  ]
}
#badge("NEW")
#badge("URGENT", color:
  rgb("#c0392b"))
#badge("OK", color: rgb("#27ae60"))
#badge("Draft", color: luma(180), text-
  color: luma(50))
```

NEW

URGENT

OK

Draft

When you call the function:

- `#badge("NEW")` → uses the default colors
- `#badge("URGENT", color: red)` → explicit color, the rest default
- The order of **named parameters** doesn't matter

17.4. Functions that accept content

The most idiomatic way in Typst to pass content to a function is to use square brackets `[...]` directly after the call:

► Function that accepts content

TYPST CODE

```
#let colored-box(color, body) = block(
  fill: color.lighten(70%),
  stroke: (left: 4pt + color),
  inset: (x: 1em, y: 0.8em),
  radius: 4pt,
  width: 100%,
)[#body]
#colored-box(blue)[
  This is a blue box with _italic_ and
  *bold*.
```

RESULT

This is a blue box with *italic* and **bold**.

This is a red box.


```

}
#colored-box(red)[
  This is a red box.
]

```

And that explains how I made these special boxes, with defined functions!

17.4.1. The special syntax with a body parameter

For functions where content is the main argument, Typst allows a shorthand syntax. The **final parameter** called `body` (or any other name you use as the last positional parameter) can be passed with `[...]` outside the parentheses:

```

#let highlightit(color: yellow, body) = highlight(
  fill: color,
  body,
)
// Normal use:
#highlightit(body: [Highlighted text])
// Shorthand syntax (body in []):
#highlightit[Highlighted text]
#highlightit(color: orange)[Orange text]

```

► Function to emphasize text

TYPST CODE

```

#let emphasis(body) = text(
  fill: rgb("#1a5276"),
  style: "italic",
  weight: "bold",
)[#body]
In the text you can
#emphasis[emphasize] certain words
in a #emphasis[consistent] way
throughout the document.

```

RESULT

In the text you can ***emphasize*** certain words in a ***consistent*** way throughout the document.

17.5. Functions that return values

Functions can return any type: numbers, strings, content, arrays, dictionaries...

► Function that returns a dictionary

TYPST CODE

```
// A function that creates a color
palette
#let palette(base-color) = (
  light: base-color.lighten(60%),
  normal: base-color,
  dark: base-color.darken(30%),
)
#let blue-pal = palette(rgb("#2471a3"))
#block(fill: blue-pal.light, inset: 0.5em)
[Light]
#block(fill: blue-pal.normal, inset:
0.5em)[
  #text(fill: gray)[Normal]
]
#block(fill: blue-pal.dark, inset: 0.5em)[
  #text(fill: white)[Dark]
]
```

RESULT

Light



[Normal]

Dark

17.6. Arrow functions (closures)

Typst supports “anonymous” functions — functions without a name, defined on the fly. They are used a lot with `map`, `filter`, etc.:

```
// Anonymous function: parameter => expression
#let double = n => n * 2
#double(5) // 10
// Used inline with map:
#(1, 2, 3, 4, 5).map(n => n * n)
// (1, 4, 9, 16, 25)
// With multiple parameters: use braces
#let sum = (a, b) => a + b
#sum(3, 4) // 7
```

17.7. Recursion

A function can call itself (recursion). Useful for nested structures:

► Recursive function

TYPST CODE

```
#let factorial(n) = {  
  if n <= 1 { 1 }  
  else { n * factorial(n - 1) }  
}  
- 5! = #factorial(5)  
- 10! = #factorial(10)
```

RESULT

- 5! = 120
- 10! = 3628800

17.8. Complete example: a card template

Creating a reusable template for letters or cards:

► Reusable card template

TYPST CODE

```
#let card(title: "Title", color: C.blu,  
body) = block(  
  width: 100%,  
  radius: 8pt,  
  stroke: 1pt + luma(210),  
  clip: true,  
  below: 1em,  
) [  
  #block(  
    fill: color,  
    width: 100%,  
    inset: (x: 1em, y: 0.6em),  
  ) [  
    #text(fill: white, weight: "bold") [#title]  
  ]  
  #block(  
    fill: white,  
    width: 100%,  
    inset: (x: 1em, y: 0.9em),  
  ) [#body]  
]  
#card(title: " Important Note") [  
  Remember to save the file before  
  closing.  
]  
#card(title: " Completed", color:
```

RESULT

Important Note

Remember to save the file before closing.

Completed

The document was delivered successfully.

```
rgb("#1e8449"))[  
  The document was delivered  
  successfully.  
]
```

18. The set rules

The `set` rules are one of Typst's most powerful and elegant mechanisms. They let you change the **default parameters** of any element, once and for all.

18.1. What is a set rule?

Imagine you want all the text in your document to be 12pt in the "Georgia" font. You could write `#text(size: 12pt, font: "Georgia") [...]` around every single paragraph, right? Sure, to waste time... it would be madness, we use Typst to optimize! With `set`, you say it just once:

```
#set text(size: 12pt, font: "Georgia")
```

And from that point on, **all** text uses those settings.

set rule — A `set` rule changes the default parameters of a function (or element) for everything that comes after. Syntax: `#set function(parameter: value)`.

18.2. set text — text

```
#set text(  
  font: "Linux Libertine", // Font name  
  size: 11pt, // Size  
  fill: luma(30), // Color (almost black)  
  weight: "regular", // "bold", "light", "regular", ...  
  style: "normal", // "italic", "oblique"  
  tracking: 0pt, // Letter spacing  
  spacing: 100%, // Word spacing  
  lang: "en", // Language (for hyphenation, etc.)  
  hyphenate: true, // Automatic hyphenation  
)
```

► set text**TYPST CODE**

```
#set text(font: "Roboto", size: 13pt, fill:
rgb("#2c3e50"))
This text uses Roboto at 13pt in dark
blue.
All the paragraphs that follow will use
this style.
```

RESULT

This text uses Roboto at 13pt in dark blue. All the paragraphs that follow will use this style.

18.3. set par — paragraphs

```
#set par(
  justify: true, // Justification (left and right alignment)
  leading: 0.75em, // Leading (space between lines)
  spacing: 1.2em, // Space between paragraphs
  first-line-indent: 1.5em, // First-line indent
  hanging-indent: 0pt, // Hanging indent
)
```

► set par**TYPST CODE**

```
#set par(justify: true, leading: 1.2em,
first-line-indent: 1.5em)
This is the first paragraph with
justification enabled
and increased leading. Notice how the
text spreads
evenly across the full width of the
column.
This is the second paragraph. The first
line has an indent
like in a traditional book. The wider
leading makes
the text more readable.
```

RESULT

This is the first paragraph with justification enabled and increased leading. Notice how the text spreads evenly across the full width of the column. This is the second paragraph. The first line has an indent like in a traditional book. The wider leading makes the text more readable.

18.4. set heading – headings

```
#set heading(  
  numbering: "1.1.", // Numbering scheme  
  outlined: true, // Appears in the TOC?  
)
```

18.5. set page – the page

```
#set page(  
  paper: "a4", // Paper format  
  margin: ( // Margins  
    top: 2.5cm,  
    bottom: 2.5cm,  
    left: 3cm,  
    right: 2.5cm,  
  ),  
  numbering: "1", // Page number format  
  number-align: center, // Number position: center, right, left  
  columns: 1, // Number of columns  
  fill: white, // Page background color  
  flipped: false, // Orientation (true = landscape)  
)
```

Available paper formats:

| Format | Code | Format | Code |
|--------|------|--------------|---------------------|
| A0 | "a0" | A5 | "a5" |
| A1 | "a1" | A6 | "a6" |
| A2 | "a2" | US Letter | "us-letter" |
| A3 | "a3" | US Legal | "us-legal" |
| A4 | "a4" | Presentation | "presentation-16-9" |

18.6. set list and set enum

```
// Bulleted list  
#set list(  
  marker: "→", // Custom marker  
  indent: 1.5em, // Indent  
  spacing: 0.6em, // Space between items  
)  
// Numbered list
```

```
#set enum(
  numbering: "1.", // Numbering scheme
  indent: 1.5em,
  spacing: 0.6em,
)
```

18.7. set table – tables

```
#set table(
  stroke: 0.5pt + luma(200), // Default border
  inset: 0.5em, // Cell padding
  align: left, // Alignment
  fill: (col, row) => // Color function
    if calc.odd(row) { luma(245) } else { white },
)
```

18.8. set figure – figures

```
#set figure(
  supplement: "Figure", // Text before the number
  gap: 0.8em, // Space between content and caption
  numbering: "1", // Numbering scheme
  placement: none, // Placement
)
```

18.9. set math.equation – equations

```
#set math.equation(
  numbering: "(1)", // Equations get numbered
  supplement: "Eq.", // Text in the reference
)
```

18.10. The scope of set rules

The `set` rules have a **scope**: they apply from where they are defined onward, or only inside the `{ }` or `||` block where they are located.

► Scope of set rules

TYPST CODE

```
Normal text in 11pt.  
#{  
  set text(size: 14pt, fill: red)  
  [This text is big and red.]  
  [This one too!]  
}  
Normal text again in 11pt.
```

RESULT

Normal text. **This text is big and red. This one too!** Normal text again.

18.11. set inside functions

You can use `set` inside a function to apply style only to that function's content:

► set inside a function

TYPST CODE

```
#let important-block(body) = {  
  set text(size: 12pt)  
  set par(leading: 1em)  
  block(  
    fill: rgb("#fef9e7"),  
    inset: 1.2em,  
    radius: 6pt,  
    stroke: 0.5pt + luma(210),  
  )[#body]  
}  
#important-block[  
  This content has a custom internal  
  style.  
  The text is slightly bigger and with  
  more leading.  
]  
This text is back to normal.
```

RESULT

This content has a custom style.
The text has more leading and a
pale-yellow background.

This text is back to normal.

19. The show rules

The `show` rules are more powerful than `set`: they don't just change the **parameters** of an element, they **completely transform its appearance**, intercepting the element before it's rendered in the PDF.

show rule — A `show` rule intercepts an element (or all elements of a certain type) and transforms it into something different. Syntax: `#show selector: transformation`.

19.1. The difference between set and show

```
// set: modifies parameters → changes properties
#set text(size: 14pt) // The text becomes bigger
// show: transforms the element → can change structure
#show strong: it => text(fill: red, it.body) // Bold becomes red
```

19.2. Transforming bold

► show strong

TYPST CODE

```
// Make bold text blue
#show strong: it => text(fill:
  rgb("#1a3a5c"), it.body)
This is normal text.
This is *bold text*, now it appears
blue.
*This sentence too* is in blue bold.
```

RESULT

This is normal text. This is **bold text**,
now it appears blue. **This sentence
too is in blue bold.**

19.3. Transforming links

► show link

TYPST CODE

```
#show link: it => {  
  set text(fill: rgb("#2471a3"))  
  underline(stroke: (paint:  
    rgb("#2471a3"), dash: "dashed"), it)  
}  
Visit #link("https://typst.app")[Typst]  
or  
#link("https://github.com/typst/typst")  
[GitHub].
```

RESULT

Visit [Typst](https://typst.app) or [GitHub](https://github.com/typst/typst).

19.4. Transforming inline code

► show raw (inline code)

TYPST CODE

```
#show raw.where(block: false): it =>  
box(  
  fill: rgb("#fff3e0"),  
  inset: (x: 4pt, y: 2pt),  
  radius: 3pt,  
  stroke: 0.5pt + rgb("#ff8f00"),  
)[  
  #set text(fill: rgb("#e65100"), font:  
    "Courier New")  
  #it  
]  
Use #raw("npm install") to install the  
packages.  
The variable #raw("n") is declared with  
#raw("let n = 0").
```

RESULT

Use `npm install` to install the pack-
ages. The variable `n` is declared
with `let n = 0`.

19.5. Selectors: .where()

You can apply the `show` rule only to a subset of elements using `.where()`:

```
// Only level-1 headings  
#show heading.where(level: 1): it => ...
```

```
// Only block code (not inline)
#show raw.where(block: true): it => ...
// Only figures that contain images
#show figure.where(kind: image): it => ...
// Only figures that contain tables
#show figure.where(kind: table): it => ...
```

19.6. Transforming headings

► Custom show heading

TYPST CODE

```
#show heading.where(level: 2): it => {
  v(1em)
  block[
    #set text(fill: rgb("#7d3c98"), size:
    14pt)
    #it.body
    #v(-0.2em)
    #line(length: 100%, stroke: 2pt +
    rgb("#7d3c98"))
  ]
  v(0.4em)
}
== First purple section
Text under the first section.
== Second purple section
Text under the second section.
```

RESULT

First purple section

Text under the first section.

Second purple section

Text under the second section.

19.7. Transforming the whole document (global show)

The most powerful form: it transforms the content of the whole document by wrapping it in something:

```
// Put the whole document in a column with max-width
#show: body => block(
  width: 80%,
  inset: 2cm,
)[#body]
// Or apply global settings
#show: body => {
```

```
set text(size: 11pt, font: "Linux Libertine")
set par(justify: true)
body
}
```

This is the basis of all Typst **templates**.

19.8. Transforming figures

► Custom show figure

```
#show figure.where(kind: image): it => {
  align(center)[
    #block(
      stroke: 2pt + luma(200),
      radius: 6pt,
      clip: true,
      inset: 0pt,
    )[#it.body]
    #v(0.3em)
    #text(size: 9pt, fill: luma(100), style: "italic")[
      #it.supplement #context it.counter.display(): #it.caption.body
    ]
  ]
}
```

19.9. show for text substitution

You can use `show` to automatically replace text strings with other content:

► show for text substitution

TYPST CODE

```
// Every time you write "Typst", it adds
the logo
#show "Typst": [*Typst*#box(
  baseline: -0.25em,
  height: 0.8em,
)][#text(size: 0.6em, fill:
```

RESULT

Welcome to the world of **Typst**. **Typst** is a powerful language. We use **Typst** every day.

```
rgb("#239dad"), baseline: 0pt)[™]  
]]  
Welcome to the world of Typst.  
Typst is a powerful language.
```

20. Colors and graphics

20.1. Color systems

Typst supports several ways to specify colors.

20.1.1. RGB

The most common: three values (Red, Green, Blue) from 0 to 255:

```
#rgb(255, 0, 0) // Pure red  
#rgb(0, 0, 255) // Pure blue  
#rgb(128, 128, 128) // Medium gray  
// With a hex string (as in CSS):  
#rgb("#ff0000") // Red  
#rgb("#1a3a5c") // Dark blue  
#rgb("#27ae60") // Green
```

20.1.2. Grayscale

```
#luma(0) // Black  
#luma(128) // 50% gray  
#luma(255) // White
```

20.1.3. OKLCH (modern, perceptually uniform)

```
// oklch(lightness, chroma, hue)  
#oklch(60%, 0.2, 240deg) // Medium blue  
#oklch(80%, 0.15, 30deg) // Light orange
```

20.1.4. Predefined colors

black white gray silver
red maroon orange yellow
olive green teal navy
blue aqua purple fuchsia
lime lime ...

Use them with the `color.` prefix:

```
color.red color.blue color.green  
// Or directly (no prefix in color contexts):  
red blue green
```

20.2. Manipulating colors

```
// Lighten  
#color.red.lighten(30%) // Red 30% lighter  
#rgb("#1a3a5c").lighten(50%)  
// Darken  
#color.blue.darken(20%)  
// Transparency (0% = opaque, 100% = invisible)  
#color.green.transparentize(60%)  
// Mix two colors  
#color.mix(red, blue) // 50/50 mix  
#color.mix((red, 30%), (blue, 70%)) // Weighted mix  
// Invert  
#color.red.negate()
```

► Color gradient

TYPST CODE

```
#let base = rgb("#2471a3")  
#for p in (0%, 20%, 40%, 60%, 80%) {  
  block(  
    fill: base.lighten(p),  
    width: 100%,  
    inset: 0.4em,  
    radius: 3pt,  
    below: 0.3em,  
  )  
  #text(fill: if p > 50% { black } else  
    { white })  
  Lightened by #p  
}
```

RESULT

Lightened by 0%
Lightened by 20%
Lightened by 40%
Lightened by 60%
Lightened by 80%

```
}  
}  
}
```

20.3. Gradients

Gradients are smooth transitions between two or more colors.

20.3.1. Linear gradient

```
gradient.linear(color1, color2)  
gradient.linear(color1, color2, angle: 45deg)  
gradient.linear(  
  (color1, 0%), // Color at the start  
  (color2, 60%), // Color at 60%  
  (color3, 100%), // Color at the end  
  angle: 135deg,  
)
```

20.3.2. Radial gradient

I'll show them to you, but don't use them, everyone hates them.

```
gradient.radial(center-color, edge-color)  
gradient.radial(  
  white,  
  blue,  
  center: (30%, 40%), // Shifted center  
  radius: 60%,  
)
```

► Block with gradient

TYPST CODE

```
#block(  
  width: 100%,  
  height: 3cm,  
  radius: 8pt,  
  fill: gradient.linear(  
    rgb("#1a5276"), rgb("#2980b9"),  
  )  
)
```

RESULT



Blue gradient

```
rgb("#aed6f1"),
  angle: 135deg,
),
)[
  #align(center + horizon)[
    #text(fill: white, size: 16pt, weight:
"bold")[
    Blue gradient
  ]
]
]
```

20.4. Drawing basic shapes

20.4.1. Line

```
#line(length: 100%, stroke: 1pt + black) // Horizontal
#line(start: (0pt, 0pt), end: (2cm, 1cm), stroke: red) // Diagonal
```

20.4.2. Rectangle

```
#rect(
  width: 4cm,
  height: 2cm,
  fill: rgb("#eaf4fb"),
  stroke: 1pt + rgb("#2471a3"),
  radius: 5pt,
)
```

20.4.3. Circle and ellipse

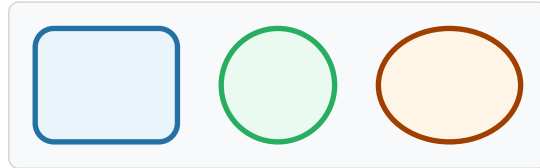
```
#circle(radius: 1cm, fill: red)
#ellipse(width: 4cm, height: 2cm, fill: blue)
```

► Geometric shapes

TYPST CODE

RESULT


```
#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 1em,
  align: center,
  rect(width: 100%, height: 1.5cm, fill:
    rgb("#eaf4fb"),
    stroke: 2pt + C.blu2, radius: 8pt),
  circle(radius: 0.75cm, fill:
    rgb("#eafaf1"),
    stroke: 2pt + C.verde2),
  ellipse(width: 100%, height: 1.5cm,
    fill: rgb("#fef5e7"),
    stroke: 2pt + C.arancio),
)
```



And that's how I created the cover of this handbook. My advice is to experiment a bit with hex and rgb (there are wonderful palettes online, just grab those).

20.5. The stroke

The `stroke` defines a shape's border. You can specify it simply or in great detail:

```
// Simple (thickness + color)
stroke: 2pt + black
// Detailed
stroke: stroke(
  thickness: 2pt,
  paint: rgb("#1a3a5c"),
  cap: "round", // "butt", "round", "square"
  join: "round", // "miter", "round", "bevel"
  dash: "dashed", // "solid", "dashed", "dotted", "dot-dash"
)
// Left side only (for boxes)
stroke: (left: 4pt + blue)
// Different borders for each side
stroke: (
  top: 2pt + black,
  bottom: 1pt + gray,
  left: none,
  right: none,
)
```

21. Boxes and blocks

`box` and `block` are the two fundamental containers in Typst. Understanding the difference is essential for controlling layout.

21.1. box vs block: the difference

| | box | block |
|--------|---------------------------|------------------------------|
| Type | Inline (in the text flow) | Block (takes the full width) |
| Use | Inside a line of text | Separate element |
| Size | Fits the content | 100% width by default |
| Wraps? | No (stays in line) | Yes (new line) |

► box vs block

TYPST CODE

```
Text with #box(fill: yellow, inset: 3pt)[a
highlighted word]
in the middle of the sentence.
#block(fill: rgb("#eaf4fb"), inset: 1em,
radius: 5pt)[
  This is a separate block that takes up
  all the available width.
]
Text that resumes after the block.
```

RESULT

Text with a highlighted word in the middle of the sentence.

This is a separate block that takes up all the available width.

Text that resumes after the block.

21.2. Parameters of box and block

Both share these main parameters:

```
#block(
  // Dimensions
  width: 80%, // Width (length, ratio, or auto)
  height: auto, // Height

  // Internal spacing
  inset: 1em, // Same on all sides
  inset: (x: 1em, y: 0.5em), // Horizontal and vertical
```

```

inset: (top: 0.5em, bottom: 0.5em, left: 1em, right: 1em),

// External spacing (block only)
above: 1em, // Space above
below: 1em, // Space below

// Appearance
fill: white, // Background color
stroke: 1pt + black, // Border
radius: 5pt, // Rounded corners

// Behavior
clip: false, // Clip overflowing content?
breakable: true, // Breaks across pages? (block only)
)

```

21.2.1. Custom rounded corners

```

#block(
  radius: (top-left: 10pt, top-right: 10pt,
    bottom-left: 0pt, bottom-right: 0pt),
)

```

21.2.2. Detailed inset

```

inset: (top: 0.5em, bottom: 0.5em, left: 1.5em, right: 0.5em)
inset: (x: 1em, y: 0.5em) // Shorthand

```

21.3. Preventing page breaks

If you don't want a block to be split across two pages:

```

#block(breakable: false)[
  This important table must not be split
  in half across two pages.

  #table(...)
]

```

21.4. Positioning with place()

`#place()` positions an element **absolutely** relative to its container, without moving the text flow:

```
#place(
  top + right, // Where to position
  dx: -1cm, // Horizontal offset
  dy: 0.5cm, // Vertical offset
  float: false, // true = floats in the text area
)[
  #badge("DRAFT")
]
```

Available positions:

- Horizontal: `left`, `center`, `right`
- Vertical: `top`, `horizon`, `bottom`
- Combinations: `top + left`, `center + horizon`, etc.

► Positioning with place()

TYPST CODE

```
#block(width: 100%, height: 3cm,
  fill: rgb("#f4f6f7"), stroke: 1pt +
  luma(200),
  radius: 8pt)[
  #place(top + left, dx: 0.5em, dy:
  0.3em)[
    #text(size: 8pt, fill: luma(120))[Top
    left]
  ]
  #place(center + horizon)[
    #text(weight: "bold")[Center]
  ]
  #place(bottom + right, dx: -0.5em, dy:
  -0.3em)[
    #text(size: 8pt, fill: luma(120))
    [Bottom right]
  ]
]
```

RESULT

Top left

Center

Bottom right

21.5. Stack: stacking elements

`#stack()` stacks elements on top of each other or side by side:

```
// Stack vertically (default)
#stack(dir: ttb, spacing: 1em,
  block(fill: red, width: 100%, height: 1cm)[] ,
  block(fill: green, width: 100%, height: 1cm)[] ,
```

```
)
// Stack horizontally
#stack(dir: ltr, spacing: 0.5em,
  rect(width: 2cm, fill: red),
  rect(width: 3cm, fill: blue),
)
```

21.6. Alignment

`#align()` aligns content inside its container:

```
#align(center)[Centered]
#align(right)[Right]
#align(left)[Left]
#align(center + top)[Center top]
#align(right + bottom)[Right bottom]
```

► align() centered

TYPST CODE

```
#block(width: 100%, height: 2cm, fill:
  luma(245),
  stroke: 0.5pt + luma(200), radius: 5pt)
[
  #align(center + horizon)[
    This text is centered horizontally and
    vertically.
  ]
]
```

RESULT

This text is centered horizontally and vertically.

22. Advanced layout

22.1. Headers and footers

Headers (`header`) and footers (`footer`) are defined inside `#set page()`.

22.1.1. Simple header

```
#set page(  
  header: [  
    My Document  
    #h(1fr)  
    Chapter 1  
  ],  
)
```

22.1.2. Header with page number

```
#set page(  
  header: context {  
    [My Book]  
    h(1fr)  
    counter(page).display()  
  },  
  footer: none,  
)
```

22.1.3. Context and document awareness

To access dynamic document information (page number, current section, counters) you must use the `context` block. It's how Typst manages the information that changes throughout the document:

```
#set page(  
  numbering: "1",  
  header: context {  
    // Current page number  
    let pn = counter(page).get().first()  
    // Show the header only after the first page  
    if pn > 1 {  
      grid(  
        columns: (1fr, auto),  
        [My Document],  
        counter(page).display("1 / 1"),  
        both: true // "current page / total pages"  
      ),  
    }  
  },  
  line(length: 100%, stroke: 0.5pt + luma(200))  
)
```

22.1.4. Header with current section title

```
#set page(
  header: context {
    let headings = query(heading.where(level: 1).before(here()))
    if headings.len() > 0 {
      let current = headings.last()
      [#current.body_]
      h(1fr)
      counter(page).display()
    }
  },
)
```

22.2. Page numbering

A bit like for a bulleted list, there are different page-numbering formats:

| String | Result | Typical use |
|---------|---------------|----------------------|
| "1" | 1, 2, 3... | Main body |
| "i" | i, ii, iii... | Preface/appendix |
| "I" | I, II, III... | Roman numerals |
| "a" | a, b, c... | Appendices |
| "A" | A, B, C... | Uppercase appendices |
| "1 / 1" | 3 / 12 | Page of total |
| "- 1 -" | - 3 - | Magazine style |

22.2.1. Changing numbering within the document

```
// Preface: Roman numerals
#set page(numbering: "i")
[... preface ...]
// Reset and switch to Arabic numerals for the body
#counter(page).update(1)
#set page(numbering: "1")
[... chapters ...]
```

22.3. Columns

To put text in multiple columns:

```
// Two columns for the whole document
#set page(columns: 2)
```

```
// Two columns for a specific block
#columns(2)[
  This text is in two columns.
  Here there's enough text to see
  how the column layout works.
]
```

22.3.1. Skipping a column

```
#columns(2)[
  Text in the first column.
  #colbreak() // Jump to the next column
  Text in the second column.
]
```

► Two-column layout

TYPST CODE

```
#columns(2, gutter: 1.5em)[
  == Side-by-side columns
  This text flows naturally into the
  first column. When the first is full,
  the text continues into the second.
  Columns are very useful for bulletins,
  newsletters, or magazine-style
  articles.
  #colbreak()
  == Second column
  With #raw("colbreak()") you can force
  the text
  to jump immediately to the next
  column,
  useful for starting a new section.
]
```

RESULT

Side-by-side

columns This text flows naturally into the first column. When the first is full, the text continues into the second. Columns are very useful for bulletins, newsletters, or magazine-style articles.

Second column

With `colbreak()` you can force the text to jump to the next column, useful for a new section.

22.4. Special pages

22.4.1. Page with different margins

```
// Larger margins for the cover
#page(
```



```
margin: 0pt,  
header: none,  
footer: none,  
numbering: none,  
)[  
  // Full-page cover content  
]
```

22.4.2. Landscape page

```
#page(flipped: true)[  
  // Content on a landscape page  
  // Useful for very wide tables  
]
```

23. Grid layout with grid()

`#grid()` is the most versatile function for structured layout. It lets you arrange elements in rows and columns with precise control.

23.1. Basic syntax

```
#grid(  
  columns: (...), // Column definition  
  rows: (...), // Row definition (optional)  
  gutter: ..., // Space between cells  
  align: ..., // Alignment  
  [Cell 1], [Cell 2], [Cell 3],  
  [Cell 4], [Cell 5], [Cell 6],  
)
```

23.2. Defining columns

Identical to `table`:

```
columns: 3 // 3 equal columns  
columns: (1fr, 2fr, 1fr) // Proportions  
columns: (auto, 1fr) // First fits, second fills  
columns: (3cm, 4cm, 5cm) // Fixed sizes
```

23.3. Practical layout examples

23.3.1. Cover layout (icon + text)

► Icon + text layout

TYPST CODE

```
#grid(  
  columns: (auto, 1fr),  
  gutter: 1em,  
  align: (center + horizon, left),  
  block(  
    fill: C.blu, width: 3cm, height: 3cm,  
    radius: 8pt,  
  ),  
  block[  
    #text(size: 18pt, weight: "bold")[My  
Document]  
    #v(0.3em)  
    #text(fill: luma(100))[Author: Mario  
Rossi]  
    #v(0.3em)  
    #text(fill: luma(100))[Date: June  
2026]  
  ],  
)
```

RESULT



My Document

Author: Mario Rossi

Date: June 2026

23.3.2. Three info-column layout

► Three-column cards

TYPST CODE

```
#let info-box(title, body) = block(  
  fill: C.sfondo, inset: 1em, radius: 6pt,  
  stroke: 1pt + luma(215),  
) [  
  #align(center) [  
    #text(weight: "bold", fill: C.blu)[#title]  
  ]  
  #v(0.5em)  
  #body
```

RESULT

Fast

Instant
compila-
tion

Beauti- ful

Profes-
sional ty-
pogra-
phy

Power- ful

Inte-
grated
scripting

```
]
#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 1em,
  info-box("Fast")[Instant compilation],
  info-box("Beautiful")[Professional
typography],
  info-box("Powerful")[Integrated
scripting],
)
```

23.3.3. Grid with spanning cells

```
#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 0.5em,
  // Cell spanning 2 columns
  grid.cell(colspan: 2)[Large cell],
  [Normal cell],
  [A], [B], [C],
)
```

23.4. The layout() function

For adaptive layouts that depend on the available size:

```
#layout(size => {
  // 'size' contains the available width and height
  let cols = if size.width > 400pt { 3 } else { 1 }
  grid(columns: cols, ...)
})
```

24. Multiple files and modules

When the document gets long or you want to reuse styles across multiple documents, it's worth organizing it into multiple files.

24.1. `#include` — including content

`#include "file.typ"` inserts the **content** of another file exactly where the instruction is. It's like pasting the file's content at the current point:

```
// Folder structure:  
main-document.typ  
chapters/  
  ch01-introduction.typ  
  ch02-methods.typ  
  ch03-results.typ  
styles/  
  configuration.typ
```

```
// main-document.typ  
#include "styles/configuration.typ"  
= My Document  
#include "chapters/ch01-introduction.typ"  
#include "chapters/ch02-methods.typ"  
#include "chapters/ch03-results.typ"
```

24.1.1. Advantages of `#include`

- Files are separate → easier to find things
- You can work on one chapter at a time
- Collaborators work on different files → fewer conflicts
- You can reorder chapters simply by moving the lines

24.2. `#import` — importing functions and variables

`#import` lets you **import** definitions (functions, variables, constants) from another file, without including its textual content:

```
// In the file 'styles/macros.typ':  
#let note(body) = block(fill: blue.lighten(80%), ...)[#body]  
#let warning(body) = block(fill: red.lighten(80%), ...)[#body]  
#let main-color = rgb("1a3a5c")
```

```
// In the main file:  
#import "styles/macros.typ": note, warning, main-color  
// Now you can use:  
#note[This is a note.]  
#warning[This is a warning.]
```

24.2.1. Importing everything with *

```
#import "styles/macros.typ": *
// Imports all public definitions
```

24.2.2. Importing with aliases

```
#import "styles/macros.typ": note as n, warning as warn
// Use #n[] instead of #note[], and #warn[] instead of #warning[]
```

24.2.3. Difference between import and include

| | #include | #import |
|----------------|----------------------------|--|
| What it does | Inserts the file's content | Imports definitions (functions, variables) |
| Produces text? | Yes | No |
| Typical use | Chapters, sections | Style libraries, macros |
| Accesses | The whole file is rendered | Only what you export |

24.3. Organizing a large project

Here's a typical structure for a thesis or a book:

```
thesis/
├── main.typ ← Main file
├── config/
│   ├── page.typ ← set page(), set text(), etc.
│   ├── styles.typ ← show rules, heading styles
│   └── macros.typ ← Custom functions
├── chapters/
│   ├── 00-title-page.typ
│   ├── 01-introduction.typ
│   ├── 02-theory.typ
│   ├── 03-methods.typ
│   ├── 04-results.typ
│   ├── 05-conclusions.typ
│   └── appendices.typ
├── images/
│   ├── logo.svg
│   └── charts/
│       ├── fig01.png
│       └── fig02.png
└── bib/
    └── references.bib
```

```
// main.typ
#import "config/macros.typ": *
#include "config/page.typ"
#include "config/styles.typ"
#include "chapters/00-title-page.typ"
#outline(depth: 3)
#counter(page).update(1)
#include "chapters/01-introduction.typ"
#include "chapters/02-theory.typ"
// ...
#bibliography("bib/references.bib")
```

24.4. Shared variables across files

You can define global variables in a configuration file and import them everywhere:

```
// config/variables.typ
#let author = "Mario Rossi"
#let title = "My Thesis"
#let year = 2026
#let university = "University of Milan"
#let advisor = "Prof. Luigi Verdi"
#let C = ( // Color palette
  blu: rgb("#1a3a5c"),
  accento: rgb("#2471a3"),
)
```

```
// In any chapter:
#import "config/variables.typ": author, year, C
Thesis by #author, year #year.
```

25. Packages

Typst's ecosystem grows every day thanks to community packages. Packages add features not present in the standard library.

25.1. Where to find packages

The official repository is **Typst Universe**: <https://typst.app/universe> You can search by category: math, presentations, CVs, tables, diagrams, graphics, citations...

25.2. How to import a package

```
#import "@preview/package-name:version": function1, function2
```

The `@preview/` prefix indicates community packages.

Note

In the typst.app online editor, packages are downloaded automatically. If you use Typst locally, they are downloaded automatically the first time you use them and then cached.

25.3. Main packages

25.3.1. showybox — Styled boxes

```
#import "@preview/showybox:2.0.1": showybox
#showybox(
  title: "Pythagorean theorem",
  frame: (title-color: blue, border-color: blue),
)[
  In a right triangle, the square of the hypotenuse is
  equal to the sum of the squares of the legs:
  $ a^2 + b^2 = c^2 $
]
```

25.3.2. codly — Code with line numbers

```
#import "@preview/codly:0.2.0": *
#show: codly-init.with()
// Now code blocks have line numbers and advanced styling
```

25.3.3. cetz — Vector drawing

```
#import "@preview/cetz:0.3.1": canvas, draw
#canvas({
  import draw: *
  circle((0, 0), radius: 1)
  line((0, 0), (1, 0))
  content((0, -1.5), [Circle])
})
```

25.3.4. fletcher — Diagrams and arrows

```
#import "@preview/fletcher:0.5.1" as fletcher: diagram, node, edge
#diagram(
  node((0,0), [State A]),
  edge("->", [transformation]),
  node((2,0), [State B]),
)
```

25.3.5. physica — Physics notation

```
#import "@preview/physica:0.9.3": *
$dv(f, x)$ // df/dx
$pdv(f, x, y)$ // Partial derivative
$laplacian(f)$ // Laplacian
$grad f$ // Gradient
$div bold(F)$ // Divergence
```

25.3.6. polylux — Presentations

```
#import "@preview/polylux:0.3.1": *
#set page(paper: "presentation-16-9")
#set text(size: 20pt)
#polylux-slide[
  = Slide title
  Content of the first slide.
]
#polylux-slide[
  == Second slide
  #pause // Pause for animations
  This appears after the pause.
]
```

25.3.7. tablex — Advanced tables

```
#import "@preview/tablex:0.0.8": tablex, cellx
#tablex(
  columns: 3,
  // Cells with simpler colspan/rowspan
  cellx(colspan: 2)[Merged],
  [Normal],
)
```


25.3.8. lovelace – Pseudocode and algorithms

```
#import "@preview/lovelace:0.3.0": *
#pseudocode-list[
  *Input:* List  $A$  of  $n$  elements
  *Output:*  $A$  sorted
  + *for*  $i$  *from*  $1$  *to*  $n-1$  *do*
  + *for*  $j$  *from*  $0$  *to*  $n-i-1$  *do*
  + *if*  $A[j] > A[j+1]$  *then*
  + swap  $A[j]$  and  $A[j+1]$ 
]
```

25.3.9. codelst – Code listings with references

25.3.10. diagraph – Graphs with GraphViz

```
#import "@preview/diagraph:0.2.5": raw-render
#raw-render(
  digraph {
    A -> B
    B -> C
    A -> C
  })
```

25.4. How to choose the right version

On typst.app/universe each package shows the versions available. In general use **the latest stable version** (the highest number). If you update a package, check whether there are breaking changes in the documentation.

```
// Always specify the exact version:
#import "@preview/showybox:2.0.1": showybox
// Don't use wildcards: "@preview/showybox:*" doesn't work
```

26. Complete academic document

In this chapter we build from scratch a complete template for scientific articles or theses, putting together everything we've learned.

26.1. Minimal template for a thesis

```
// — CONFIGURATION —
#set document(
  title: "Thesis Title",
  author: "First Last",
)
#set page(
  paper: "a4",
  margin: (top: 3cm, bottom: 3cm, left: 3.5cm, right: 2.5cm),
  numbering: "1",
  number-align: right,
  header: context {
    let pn = counter(page).get().first()
    if pn > 1 {
      set text(size: 9pt, fill: luma(130))
      line(length: 100%, stroke: 0.4pt + luma(200))
      v(-0.8em)
      grid(
        columns: (1fr, auto),
        [Thesis Title_],
        counter(page).display(),
      )
    }
  },
)
#set text(font: "Linux Libertine", size: 11pt, lang: "en")
#set par(justify: true, leading: 0.75em, spacing: 1.2em)
#set heading(numbering: "1.1.1.")
#set math.equation(numbering: "(1)", supplement: "Eq.")
#set figure(supplement: "Figure")
// — TITLE PAGE —
#page(numbering: none, header: none)[
  #align(center)[
    // University logo (if available)
    // #image("univ-logo.svg", width: 4cm)
    // #v(1cm)
    #text(size: 14pt)[*UNIVERSITY OF ...*]
    #v(0.3em)
    #text(size: 12pt)[Department of ...]
    #v(0.3em)
    #text(size: 12pt)[Degree Program in ...]
    #v(2cm)
    #line(length: 80%, stroke: 1pt + black)
    #v(1cm)
    #text(size: 11pt, style: "italic")[Master's Thesis in ...]
    #v(0.5cm)
    #text(size: 22pt, weight: "bold")[
      Thesis Title \
      on Multiple Lines if Needed
    ]
  ]
]
```

```

]
#v(0.5cm)
#line(length: 80%, stroke: 1pt + black)
#v(2cm)
#grid(
  columns: (1fr, 1fr),
  align(left)[
    *Advisor:* \
    Prof. Luigi Verdi
    #v(0.5em)
    *Co-advisor:* \
    Dr. Carlo Bianchi
  ],
  align(right)[
    *Candidate:* \
    Mario Rossi \
    Student ID: 12345678
  ],
)
#v(1fr)
#text(size: 11pt)[Academic Year 2024/2026]
]
]

// — ABSTRACT

#page(numbering: "i", header: none)[
  #heading(level: 1, numbering: none, outlined: false)[Abstract]
  #text(style: "italic")[
    Lorem ipsum dolor sit amet, consectetur adipiscing elit.
    This is the abstract text in English...
  ]
  #v(1.5em)
  *Keywords:* Typst, typesetting, academic documents.
  #v(2em)
  #heading(level: 1, numbering: none, outlined: false)[Abstract (Italiano)]
  #text(style: "italic")[
    Questo è l'abstract in italiano...
  ]
  #v(1.5em)
  *Keywords:* Typst, impaginazione, documenti accademici.
]

// — TABLE OF CONTENTS —
#page(numbering: "i")[
  #outline(depth: 3, indent: 2em)
]
#page(numbering: "i")[
  #outline(target: figure.where(kind: image),
    title: [List of Figures])
  #v(2em)
  #outline(target: figure.where(kind: table),

```

```

title: [List of Tables]
]
// — DOCUMENT BODY —
#counter(page).update(1)
= Introduction
Text of the first chapter...
== Motivation
...
== Objectives
...
== Document structure
...
= Methodology
...
= Results
...
= Conclusions and Future Work
...
// — APPENDICES —
#heading(level: 1, numbering: none)[Appendix A — Appendix Name]
...
// — BIBLIOGRAPHY —
#heading(level: 1, numbering: none, outlined: true)[Bibliography]
#bibliography("references.bib", style: "ieee")
// Styles: "ieee", "apa", "chicago-author-date", "mla", ...

```

26.2. Numbered equations and references

```

#set math.equation(numbering: "(1)")
The Navier-Stokes equation for incompressible fluids is:
$ rho (partial bold(u))/(partial t) +
  rho (bold(u) dot nabla) bold(u) =
  -nabla p + mu nabla^2 bold(u) + bold(f)
$<eq-navier-stokes>
As shown in equation @eq-navier-stokes, the term
$rho (bold(u) dot nabla) bold(u)$ represents...

```

26.3. Bibliographic citations

To cite bibliographic references, Typst uses the `.bib` file in BibTeX format:

```

// references.bib
@article{knuth1984,
  author = {Knuth, Donald E.},
  title = {The {\TeX}book},

```

```

year = {1984},
journal = {Computers \& Typesetting},
}
@inproceedings{typst2023,
author = {Haug, Martin and Mädje, Laurenz},
title = {Typst: A Programmable Markup Language},
year = {2023},
}

```

```

// In the document:
As demonstrated by @knuth1984, the typographic quality...
Typst's approach @typst2023 improves on LaTeX...
// At the end of the document:
#bibliography("references.bib", style: "ieee")

```

26.4. Academic figures and tables

```

// Figure with a reference
#figure(
  image("charts/results.png", width: 85%),
  caption: [
    Performance comparison between the three proposed algorithms.
    The values represent the average over 100 runs.
  ],
) <fig-results>
As shown in @fig-results, the proposed algorithm
outperforms the baselines in all tested scenarios.
// Table with a reference
#figure(
  table(
    columns: (auto, auto, auto, auto),
    stroke: 0.5pt + luma(200),
    fill: (col, row) => if row == 0 { luma(230) } else { white },
    table.header(
      [*Dataset*], [*Accuracy*], [*Precision*], [*Recall*],
    ),
    [MNIST], [99.1%], [98.7%], [99.0%],
    [CIFAR-10], [87.3%], [86.9%], [87.1%],
    [ImageNet], [76.4%], [75.8%], [76.2%],
  ),
  caption: [Experimental results on the benchmark datasets.],
  kind: table,
) <tab-results>
@tab-results shows the quantitative results...

```

27. Curriculum Vitae template

A modern, professional CV in Typst.

27.1. Two-column CV template

```
// — CONFIGURATION —
#set document(title: "CV - Mario Rossi")
#set page(paper: "a4", margin: 0pt)
#set text(font: "Linux Libertine", size: 10pt, lang: "en")
#set par(justify: false, spacing: 0.6em)
// — COLORS —

#let C = (
  sinistra: rgb("#1a3a5c"),
  accento: rgb("#2471a3"),
)
// — HELPER FUNCTIONS —
#let cv-entry(title, subtitle: none, period: none, body) = {
  block(below: 0.8em)[
    #grid(
      columns: (1fr, auto),
      text(weight: "bold", fill: C.accento)[#title],
      if period != none { text(fill: luma(120), size: 9pt)[#period] },
    )
    #if subtitle != none {
      text(style: "italic")[#subtitle]
      linebreak()
    }
    #body
  ]
}
#let cv-section(title, body) = {
  v(0.5em)
  text(size: 12pt, weight: "bold", fill: C.accento)[#title]
  v(-0.3em)
  line(length: 100%, stroke: 1.5pt + C.accento)
  v(0.4em)
  body
}
#let skill(name, level) = {
  grid(
    columns: (1fr, auto),
    align: (left + horizon, right + horizon),
    gutter: 0.5em,
    text(size: 9.5pt)[#name],
```

```

box(width: 4cm)[
#stack(dir: ltr, spacing: 0pt,
..range(5).map(i =>
box(
width: 0.7cm, height: 0.35cm,
radius: 2pt,
inset: 0pt,
fill: if i < level { C.accento } else { luma(220) },
)
)
)
],
)
}
// — LAYOUT: LEFT COLUMN (30%) + RIGHT (70%) —————
#grid(
columns: (30%, 70%),
// — LEFT COLUMN —————
block(
fill: C.sinistra,
height: 100%,
inset: (x: 1.5em, y: 2em),
width: 100%,
)[
// Photo placeholder (replace with image())
#align(center)[
#circle(
radius: 2cm,
fill: C.accento,
)[
#align(center + horizon)[
#text(fill: white, size: 28pt)[MR]
]
]
]
#v(1em)
// Contacts
#set text(fill: white, size: 9.5pt)
#cv-section(text(fill: luma(200))[Contacts])[
mario.rossi\@email.com \
+39 333 1234567 \
linkedin.com/in/mariorossi \
Rome, Italy
]
#v(0.5em)
// Technical skills
#cv-section(text(fill: luma(200))[Skills])[
#set text(fill: white)
*Languages*
#skill("Python", 5)

```

```

#skill("Java", 4)
#skill("Typst", 5)
#v(0.4em)
*Tools*
#skill("Git", 4)
#skill("Docker", 3)
#skill("LaTeX", 4)
]
#v(0.5em)
// Languages
#cv-section(text(fill: luma(200))[Languages])[
#set text(fill: white)
#skill("Italian", 5)
#skill("English", 4)
#skill("German", 2)
]
],
// — RIGHT COLUMN —
block(
fill: white,
height: 100%,
inset: (x: 2em, y: 2em),
width: 100%,
)[
// Name and title
#text(size: 26pt, weight: "bold", fill: C.sinistra)[Mario Rossi]
#linebreak()
#text(size: 13pt, fill: C.accento)[Software Engineer]
#v(1em)
// Profile
#cv-section("Profile")[
Software engineer with 5+ years of experience in
software development. Specialized in machine learning and
distributed systems. Passionate about open-source and
technical documentation with Typst.
]
// Work experience
#cv-section("Work Experience")[
#cv-entry(
"Senior Software Engineer",
subtitle: "Tech Company Inc. · Rome",
period: "2022 – Present",
)[
- Built recommendation systems with Python/PyTorch
- Microservices architecture on Kubernetes
- Mentoring of 3 junior developers
]
#cv-entry(
"Software Developer",
subtitle: "Innovation Startup · Milan",

```



```

period: "2019 – 2022",
)[
- Backend APIs with FastAPI and PostgreSQL
- 40% reduction in response times
]
]
// Education
#cv-section("Education")[
#cv-entry(
"Master's Degree in Computer Engineering",
subtitle: "Sapienza University of Rome",
period: "2017 – 2019",
)[
Grade: 110/110 with honors. Thesis: \Machine Learning for
the prediction of industrial failures\.
]
#cv-entry(
"Bachelor's Degree in Computer Science",
subtitle: "Sapienza University of Rome",
period: "2014 – 2017",
)[Grade: 108/110.]
]
// Certifications
#cv-section("Certifications")[
- AWS Certified Solutions Architect (2023)
- Google Professional Data Engineer (2022)
- Scrum Master Certified (2021)
]
],
)

```

27.2. Minimalist CV variant (one column)

```

#set page(paper: "a4", margin: (x: 2.5cm, y: 2.5cm))
#set text(font: "Linux Libertine", size: 11pt)
// Header
#align(center)[
#text(size: 24pt, weight: "bold")[Mario Rossi]
#linebreak()
#text(fill: luma(100))[
mario@email.com · +39 333 1234567 · Rome, Italy ·
linkedin.com/in/mariorossi
]
]
#line(length: 100%, stroke: 0.5pt)
// Sections
≡ Experience

```

```
...
= Education
...
```

28. Presentations (Slides)

Typst can create professional presentations (you know Beamer in LaTeX?) thanks to the **Polylux** package (or the more recent **Touying**).

28.1. Setup with Polylux

```
// — IMPORT


---


#import "@preview/polylux:0.3.1": *
// — PAGE CONFIGURATION


---


#set page(
  paper: "presentation-16-9",
  margin: 0pt,
)
#set text(font: "Linux Libertine", size: 22pt, lang: "en")
// — COLORS


---


#let C = (
  bg: rgb("#0d2137"),
  accent: rgb("#2471a3"),
  fg: white,
  gray: rgb("#aab7b8"),
)
// — SLIDE TEMPLATE


---


#let slide-base(body) = {
  set page(fill: C.bg)
  set text(fill: C.fg)
  polylux-slide[
    #block(
      width: 100%, height: 100%,
      inset: (x: 2cm, y: 1.5cm),
    )[#body]
  ]
}
#let slide-title(title, subtitle: none, author: none) = {
  set page(fill: C.bg)
  polylux-slide[
    #align(center + horizon)[
```

```

#block(width: 80%)[
  #if subtitle != none {
    text(fill: C.accent, size: 16pt, tracking: 3pt)[
      #upper(subtitle)
    ]
    v(0.5em)
  }
  text(size: 40pt, weight: "bold")[#title]
  v(1em)
  line(length: 40%, stroke: 2pt + C.accent)
  if author != none {
    v(0.8em)
    text(fill: C.gray, size: 18pt)[#author]
  }
]
]
]
}

#let slide-section(title) = {
  set page(fill: C.accent)
  polylux-slide[
    #align(center + horizon)[
      #text(size: 36pt, weight: "bold", fill: white)[#title]
    ]
  ]
}

// — TITLE SLIDE —
#slide-title(
  "Introduction to Typst",
  subtitle: "A Modern Typesetting Course",
  author: "Mario Rossi · June 2026",
)

// — NORMAL SLIDE —
#slide-base[
  == What is Typst?
  #v(0.5em)
  - A _modern_ markup language
  - *Instant* compilation
  - Professional typographic quality
  - Integrated scripting
  #align(right)[
    #text(fill: C.gray, size: 16pt)[typst.app]
  ]
]

// — SLIDE WITH COLUMNS —
#slide-base[
  == Typst vs LaTeX
  #grid(
    columns: (1fr, 1fr),
    gutter: 2em,

```

```

block(
  fill: white.transparentize(90%),
  inset: 1em, radius: 8pt,
)[
  *LaTeX*
  - Old (1984)
  - Slow compilation
  - Cryptic errors
],
block(
  fill: C.accent.transparentize(50%),
  inset: 1em, radius: 8pt,
)[
  *Typst*
  - Modern (2023)
  - Instant
  - Clear errors
],
)
]

// — SLIDE WITH PROGRESSIVE REVEAL —————
#slide-base[
  == The main advantages
  #uncover("1-")[ Compilation speed]
  #v(0.5em)
  #uncover("2-")[ Excellent typography]
  #v(0.5em)
  #uncover("3-")[ Powerful scripting]
  #v(0.5em)
  #uncover("4-")[ Community packages]
]

// — SLIDE WITH CODE —————
#slide-base[
  == Your first document
  // Inside a slide you write normal Typst markup:
  = Hello, Typst!
  This is my first document.
  - Easy
  - Powerful
  - Beautiful
  #text(fill: C.accent)[→ A professional PDF right away!]
]

// — CLOSING SLIDE —————
#slide-base[
  #align(center + horizon)[
    #text(size: 32pt, weight: "bold")[Thank you!]
    #v(1em)
    #text(fill: C.gray, size: 18pt)[
      typst.app/universe \ github.com/typst/typst ]
    #v(1em)]

```

28.2. Progressive reveal (animations)

Polylux supports slides with elements that appear gradually:

```
// Show only on slide 2 and later
#only("2-")[Text that appears from slide 2]
// Show only on slide 3
#only(3)[Text that appears only on slide 3]
// Cover (hide) until slide 2
#uncover("2-")[Hidden, then revealed]
// Alternative: #pause adds a pause step
== Slide title
First point.
#pause
Second point (appears after a click).
#pause
Third point.
```

28.3. The Touying package (advanced alternative)

Touying is a more recent and powerful alternative to Polylux:

```
#import "@preview/touying:0.5.3": *
#import themes.university: *
#show: university-theme.with(
  aspect-ratio: "16-9",
  config-info(
    title: [The Presentation Title],
    author: [Mario Rossi],
    institution: [University of ...],
  ),
)
#title-slide()
== First Section
== Slide 1
Content of the first slide.
== Slide 2
Content of the second.
```

Touying offers:

- Many predefined themes (metropolis, dewdrop, university, ...)
- More sophisticated transitions
- Automatic handout mode
- Better support for complex content

29. Advanced topics

29.1. The context system

The `context` block is the mechanism Typst uses to manage information that depends on the position in the document.

29.1.1. Counters

Typst has counters for pages, headings, figures, equations... You can also create custom ones:

```
// Predefined counter: pages
context counter(page).get() // Current value
context counter(page).display() // Displayed as text
// Heading counter
context counter(heading).get()
// Custom counter
#let cnt-examples = counter("examples")
#cnt-examples.step() // Increment
#context cnt-examples.display() // Show the value
// Example: numbering the examples
#let numbered-example(body) = {
  cnt-examples.step()
  block(fill: luma(245), inset: 1em)[
    *Example #context cnt-examples.display()*
    #body
  ]
}
```

29.1.2. Query

`query()` inside `context` lets you search for elements in the document:

```
// Find all level-1 headings
context {
  let chaps = query(heading.where(level: 1))
  for chap in chaps {
    [• #chap.body #linebreak()]
  }
}
// Find all headings before the current position
context query(heading.where(level: 1).before(here()))
```

```
// How many figures are in the document?  
context query(figure.where(kind: image)).len()
```

29.1.3. State (global state)

State is like a global variable that can be modified during compilation:

```
#let theme = state("theme", "light") // Default: "light"  
// Modify the state  
#theme.update("dark")  
// Read the state  
#context {  
  let t = theme.get()  
  if t == "dark" { block(fill: black)[...] }  
  else { block(fill: white)[...] }  
}
```

29.2. Advanced text algorithms

29.2.1. Text with line-by-line control

```
// Fine control of the text  
#set text(  
  tracking: 0.5pt, // Letter spacing (letterspacing)  
  spacing: 110%, // Word spacing  
)  
// Text with font features  
#set text(  
  features: ("onum"), // Old-style numerals  
)
```

29.2.2. Vertical text

```
// Rotate the text  
#rotate(90deg)[Vertical text]  
// Scale the text  
#scale(x: 150%)[Widened text]
```

29.3. Patterns and repetitions

```
// Watermark  
#page(background:
```

```

place(center + horizon, rotate(-30deg,
text(size: 80pt, fill: luma(220))[DRAFT]
))
)[
  // Page content
]

```

► Tiled background

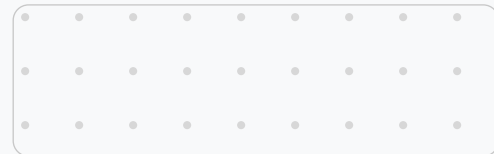
TYPST CODE

```

// Background with tiling (in old
versions it was called `pattern`)
#block(
  width: 100%,
  height: 2.5cm,
  fill: tiling(
    size: (30pt, 30pt),
    relative: "parent",
  )[
    #place(dx: 0pt, dy: 0pt)[
      #circle(radius: 2pt, fill: luma(220))
    ]
  ],
)

```

RESULT



29.4. Advanced drawing with CeTZ

For complex diagrams and vector drawings, use the `cetz` package:

```

#import "@preview/cetz:0.3.1": canvas, draw
#canvas({
  import draw: *
  // Axes
  line((-3, 0), (3, 0), mark: (end: "stealth"))
  line((0, -0.5), (0, 3), mark: (end: "stealth"))
  content((3.2, 0), $x$)
  content((0, 3.2), $y$)
  // Parabola through points
  bezier-through(
    (-2, 4), (0, 0), (2, 4),
    stroke: blue + 2pt,
  )
  // Label
  content((1.5, 2.5), [$y = x^2$, anchor: "west"])

```



```
// Points
for x in (-2, -1, 0, 1, 2) {
  circle((x, x*x), radius: 0.07, fill: blue)
}
```

29.5. Diagrams with Fletcher

Fletcher is the elegant solution for graphs and arrow diagrams:

```
#import "@preview/fletcher:0.5.1" as fletcher: diagram, node, edge
#diagram(
  cell-size: 15mm,
  node((0,0), [*Compiler*\nTypst], shape: rect),
  node((2,0), [*PDF*], shape: rect, fill: green.lighten(70%)),
  node((0,1), [.typ\nfile], fill: blue.lighten(80%)),
  edge((0,1), (0,0), "->", [reads]),
  edge((0,0), (2,0), "=>", [produces], label-pos: 0.5),
)
```

Appendix A — Common Errors

29.6. Frequent syntax errors

| Error | Wrong | Correct |
|---------------------------------|--|--|
| Superscript with multiple chars | <code>\$x^n+1\$</code> | <code>\$x^{n+1}\$</code> |
| Fraction | <code>\$a/b+c\$</code> | <code>\$(a/b)+c\$</code>
or <code>\$frac(a, b+c)\$</code> |
| String with single quotes | <code>'hello'</code> | <code>"hello"</code> |
| Comparison vs assignment | <code>if x = 5</code> | <code>if x == 5</code> |
| Infinite show rule | <code>show heading: it => heading[#it]</code> | <code>show heading: it => block[#it.body]</code> |
| Dict key with spaces | <code>(my key: v)</code> | <code>("my key": v)</code> |
| Out-of-bounds index access | <code>.at(10)</code> (on an array of 5) | Check <code>.len()</code> first |
| Tab instead of spaces | Using tabs to indent lists | Use 2 spaces |

29.7. Common error messages

| Message | Cause and solution |
|--------------------|--|
| unknown variable | You used an undefined variable. Check the name and that it's defined before use. |
| expected content | You passed a wrong value where content <code>[...]</code> is needed. Put the text in brackets. |
| unexpected keyword | Syntax error. Check the commas, brackets, and the code structure. |
| file not found | The image or included file doesn't exist at the specified path. Check the path. |
| cyclic import | Two files import each other. Restructure the dependencies. |
| stack overflow | Infinite recursion. Make sure the recursive function has a base case. |

Appendix B — Resources and Community

29.8. Official documentation

- **Typst documentation:** typst.app/docs
- **Typst Universe** (packages): typst.app/universe
- **GitHub Typst:** github.com/typst/typst

29.9. Community and support

- **Official forum:** forum.typst.app
- **Typst Discord:** Search “Typst Discord” on Discord to join the official server
- **Reddit:** reddit.com/r/typst
- **Stack Overflow:** The `typst` tag on Stack Overflow for technical questions

29.10. Complementary tools

- **Tinymist** (VS Code): the best extension for Typst
- **typst-lsp:** Language Server Protocol for any editor
- **Hayagriva:** bibliography management in YAML format
- **typst-fmt:** automatic Typst code formatter

Appendix C — Fonts and Typography

29.11. How to specify fonts

```
// Single font
#set text(font: "Linux Libertine")
// Fallback list: uses the first available
#set text(font: ("Linux Libertine", "Georgia", "Times New Roman"))
// Different fonts for serif and sans-serif
#set text(font: "Linux Libertine") // For the body
#set raw(theme: "halcyon") // For code
```

29.12. Fonts available in the online editor

Many Google Fonts are available in the typst.app editor, including:

| Family | Type | Recommended use |
|---------------------|------------|--------------------------|
| Linux Libertine | Serif | Body text, academic |
| New Computer Modern | Serif | Math, scientific |
| Libertinus Serif | Serif | Alternative to Libertine |
| Source Serif 4 | Serif | Readable body text |
| Noto Serif | Serif | Multilingual |
| Linux Biolinum | Sans-serif | Headings, interfaces |
| Noto Sans | Sans-serif | Presentations |
| Source Sans Pro | Sans-serif | Technical documentation |
| Roboto | Sans-serif | Modern, UI |
| Fira Code | Monospace | Source code |
| Cascadia Code | Monospace | Source code |
| JetBrains Mono | Monospace | Source code |

29.13. Font weights and styles

```
// Available weights (depends on the font)
#text(weight: "thin")[Thin]
#text(weight: "extralight")[ExtraLight]
```

```
#text(weight: "light")[Light]
#text(weight: "regular")[Regular]
#text(weight: "medium")[Medium]
#text(weight: "semibold")[Semibold]
#text(weight: "bold")[Bold]
#text(weight: "extrabold")[ExtraBold]
#text(weight: "black")[Black]
// Numeric weight (100–900)
#text(weight: 350)[Weight 350]
#text(weight: 700)[Weight 700]
// Styles
#text(style: "normal")[Normal]
#text(style: "italic")[Italic]
#text(style: "oblique")[Oblique]
```

29.14. Typographic measures

```
// em: relative to the current font (1em = text size)
// 1em of 12pt text = 12pt
#set par(leading: 0.75em) // Leading 75% of the size
// rem: relative to the document's base font
// Same as em if you're not in a nested context
// ex: height of the letter 'x' (about 0.5em)
// Golden rule for typography:
// - Body text: 10–12pt for A4
// - Leading: 1.2–1.5× the body
// - Margins: at least 2cm, ideally 2.5–3cm
// - Line length: 60–75 characters per line
```

29.15. Highlight and emphasis without bold

```
// Highlight (like a marker)
#highlight(fill: yellow)[highlighted text]
#highlight(fill: rgb("#a8d5ff"))[highlighted in blue]
// Underline
#underline[underlined text]
#underline(stroke: 2pt + red)[thick red underline]
#underline(stroke: (dash: "dashed"))[dashed underline]
#underline(offset: 3pt)[lower underline]
// Overline
#overline[overlined text]
// Strikethrough
#strike[struck-through text]
#strike(stroke: 2pt)[thick strikethrough]
```

► Typographic effects

TYPST CODE

You can `#highlight(fill: rgb("#fff176"))`
`[highlight]` the text,
`#underline[underline it]`, or even
`#strike[strike it through]` in style.
 With `#text(tracking: 3pt, weight: "light")`
`[letter spacing]`
 you create an elegant effect for
 headings.

RESULT

You can **highlight** the text, underline it,
 or even ~~strike it through~~ in style. With
 letter spacing you create an
 elegant effect for headings.

Appendix D — Counters and Numbering

29.16. Predefined counters

```
// Current page
context counter(page).get().first() // Number (int)
context counter(page).display() // Formatted string
// Total pages
context counter(page).final().first() // Total pages
// x/y style
context counter(page).display("1 / 1", both: true)
// Heading counter
context counter(heading).get() // Array (1, 2, 1) → ch 1.2.1
context counter(heading).display("1.1.")
// Figure counter
context counter(figure.where(kind: image)).display()
// Equation counter
context counter(math.equation).display()
```

29.17. Creating a custom counter

Very useful for numbering examples, exercises, theorems, etc.:

► Theorem counter

TYPST CODE

```
#let cnt-theorem = counter("theorem")
#let theorem(name, body) = {
  cnt-theorem.step()
  block(
    fill: rgb("#eaf4fb"),
    stroke: (left: 4pt + C.blu2),
    inset: (x: 1em, y: 0.8em),
    radius: 4pt,
    width: 100%,
  )[
    *#text(fill: C.blu2)[Theorem #context
cnt-theorem.display()
  #if name != "" [- #name]]*
  #parbreak()
  #body
  ]
}
#theorem("Pythagoras")[
  In every right triangle, the square
  of the hypotenuse equals the sum of
  the
  squares of the legs:  $a^2 + b^2 = c^2$ .
]
#theorem("")[
  Every even number greater than 2 is
  the sum
  of two prime numbers (Goldbach's
  conjecture).
]
```

RESULT

Theorem 1 – Pythagoras

In every right triangle, $a^2 + b^2 = c^2$.

Theorem 2

Every even number > 2 is the sum of two primes.

29.18. Resetting a counter

```
// Reset to 0
#cnt-theorem.update(0)
// Reset to a specific value
#counter(page).update(5) // The next page will be 6
// Reset at the start of each chapter
#show heading.where(level: 1): it => {
  cnt-theorem.update(0)
  it
}
```

29.19. Displaying the total number of pages

A common request: showing “Page X of Y”:

```
#set page(  
  footer: context [  
    #h(1fr)  
    Page #counter(page).display() of  
    #counter(page).final().first()  
  ]  
)
```

Appendix E — Ready-to-Use Templates

Below you'll find complete, ready configurations for various common scenarios. Copy, paste, and customize (don't thank me).

29.20. Template: Generic professional document

```
#set document(title: "Title", author: "First Last")  
#set page(paper: "a4", margin: (x: 2.5cm, y: 3cm),  
  numbering: "1", number-align: center)  
#set text(font: ("Linux Libertine", "Georgia"),  
  size: 11pt, lang: "en", hyphenate: true)  
#set par(justify: true, leading: 0.8em, spacing: 1.2em)  
#set heading(numbering: "1.1.")  
#show heading.where(level: 1): set text(size: 18pt)  
#show heading.where(level: 2): set text(size: 14pt)  
#show link: set text(fill: blue)  
#show raw: set text(font: ("Fira Code", "Courier New"), size: 9.5pt)
```

29.21. Template: Minimalist document

```
#set page(paper: "a4", margin: 3cm, numbering: "1")  
#set text(font: "Linux Libertine", size: 12pt, lang: "en")  
#set par(justify: true, leading: 0.9em)  
#show heading: set text(font: "Linux Biolinum")
```

29.22. Template: Technical document (manual style)

```
#set document(title: "Technical Manual", author: "Dev Team")
#set page(
  paper: "a4",
  margin: (left: 3.5cm, right: 2.5cm, top: 2.5cm, bottom: 2.5cm),
  numbering: "1",
  header: context {
    if counter(page).get().first() > 1 {
      grid(columns: (1fr, auto),
        text(size: 8.5pt, fill: luma(150))[_Technical Manual_],
        text(size: 8.5pt, fill: luma(150))[Page #counter(page).display()],
      )
      v(-0.5em)
      line(length: 100%, stroke: 0.4pt + luma(210))
    }
  },
)
#set text(font: ("Linux Libertine", "Georgia"), size: 11pt, lang: "en")
#set par(justify: true)
#set heading(numbering: "1.1.")
#show raw.where(block: true): it => block(
  fill: luma(245), inset: 1em, radius: 5pt,
  stroke: 0.5pt + luma(200), width: 100%,
)[#set text(font: ("Fira Code", "Courier New"), size: 9.5pt); #it]
```

29.23. Template: University lecture notes

```
#set document(title: "Lecture Notes", author: "Prof. Lastname")
#set page(paper: "a4",
  margin: (x: 2.5cm, y: 3cm),
  numbering: "1",
  header: context {
    let p = counter(page).get().first()
    if p > 1 {
      line(length: 100%, stroke: 0.4pt + luma(220))
      v(-0.6em)
      align(right, text(size: 8.5pt, fill: luma(130))["
Degree Program in ... · #p
"])
    }
  })
#set text(font: ("Linux Libertine", "Georgia"), size: 11pt, lang: "en")
#set par(justify: true, leading: 0.8em)
#set heading(numbering: "1.")
#set math.equation(numbering: "(1)", supplement: "Eq.")
```



```
#set figure(supplement: "Figure")
// Theorem, definition, example boxes
#let theorem(title: "Theorem", body) = block(
  fill: rgb("#eaf4fb"), stroke: (left: 4pt + blue),
  inset: 1em, radius: 4pt, width: 100%,
)[*#title.* #body]
#let definition-d(title: "Definition", body) = block(
  fill: rgb("#eafaf1"), stroke: (left: 4pt + green),
  inset: 1em, radius: 4pt, width: 100%,
)[*#title.* #body]
#let example-d(body) = block(
  fill: rgb("#fef9e7"), stroke: (left: 4pt + orange),
  inset: 1em, radius: 4pt, width: 100%,
)[*Example.* #body]
```

29.24. Template: Newsletter / Bulletin

```
#set document(title: "Newsletter")
#set page(paper: "a4", margin: (x: 1.5cm, y: 2cm),
  columns: 2, numbering: none)
#set text(font: ("Linux Libertine", "Georgia"), size: 10pt)
#set par(justify: true, leading: 0.7em)
// Full-width title (outside the columns)
// Use an initial single-column page for the header:
#page(columns: 1)[
  #align(center)[
    #block(fill: rgb("#1a3a5c"), width: 100%,
  inset: (x: 2em, y: 1.5em))[
    #text(fill: white, size: 28pt, weight: "bold")[
      MY NEWSLETTER
    ]
    #linebreak()
    #text(fill: rgb("#aed6f1"), size: 11pt)[
      Volume 1 · June 2026
    ]
  ]
  #v(0.5em)
]
// From here the text flows into 2 columns
```

29.25. Template: Academic poster (A1)

```
#set document(title: "Poster")
#set page(
  paper: "a1",
  flipped: true,
  margin: 1.5cm,
)
#set text(font: ("Linux Libertine", "Georgia"), lang: "en")
// Split into 3 columns with grid
#grid(
  columns: (1fr, 2fr, 1fr),
  gutter: 1cm,
  // Left column: methods
  block(width: 100%)[
    #heading(level: 2, numbering: none)[Methods]
    ...
  ],
  // Center column: main results
  block(width: 100%)[
    #heading(level: 2, numbering: none)[Results]
    ...
  ],
  // Right column: conclusions
  block(width: 100%)[
    #heading(level: 2, numbering: none)[Conclusions]
    ...
  ],
)
```

Appendix F (Part I) — Additional Notes: Slides and Presentations

A few small additional notes on the presentations we already discussed.

29.26. Overview of the tools

| Tool | Description | Language | Output |
|--------|-----------------------------------|----------|--------|
| Beamer | LaTeX package for academic slides | LaTeX | PDF |

| Tool | Description | Language | Output |
|-------------------|--|--------------|------------------------|
| R Markdown | Computational document with slides | R + Markdown | PDF / HTML |
| Quarto | Modern successor to R Markdown | R / Python | PDF / HTML / Reveal.js |
| knitr | Computation engine for R Markdown/Quarto | R | any |
| Polylux | Typst package for presentations | Typst | PDF |
| Touying | Advanced Typst framework for slides | Typst | PDF |

29.26.1. Beamer

Beamer is the most-used LaTeX presentation package in academic and scientific settings since 2003. It produces high typographic-quality PDFs, with perfect equations and predefined themes (Warsaw, Madrid, Metropolis, etc.).



Figure 1: Example screenshot taken from [HERE](#).

Strengths

- Excellent typographic quality
- Mature predefined themes
- Native LaTeX equations
- Very widespread in academia

Limitations

- Slow compilation (multiple passes)
- Verbose, wordy syntax
- Hard to customize themes
- No real-time preview

The basic syntax of a Beamer slide:

```
\documentclass{beamer}
\usetheme{metropolis}

\begin{document}

\begin{frame}{Slide Title}
\begin{itemize}
\item First item
\pause
\item Second item (appears after click)
\end{itemize}
\end{frame}
\end{document}
```

```
\end{document}
```

29.26.2. R Markdown

R Markdown is a format that interweaves text, R code, and output (charts, tables, results) in a single document. It can produce slides via the `ioslides`, `revealjs`, or `beamer` package.

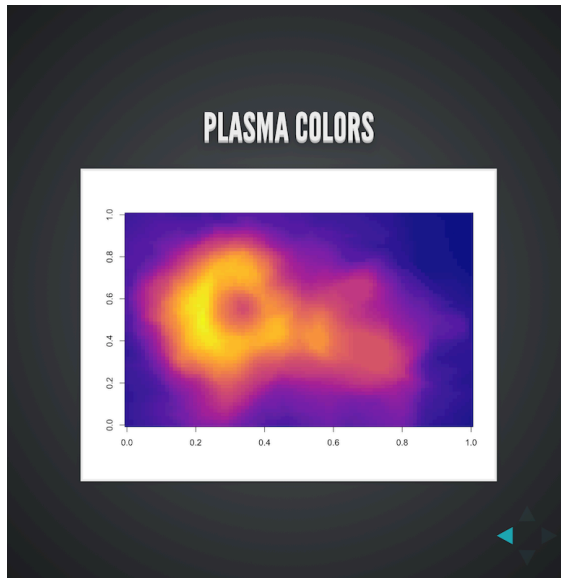


Figure 2: Example screenshot taken from [HERE](#).

Strengths

- Integrates R code and output in the document
- Automatically generated charts
- Multiple outputs: PDF, HTML, Word
- Extremely widespread in statistics and data science

Limitations

- Limited layout control
- HTML slides depend on the browser
- Not ideal for fine typography
- Dependency on Pandoc

Structure of an R Markdown file for slides:

```
---
title: "My Analysis"
output: beamer_presentation
---

## First Slide

Text and code together:

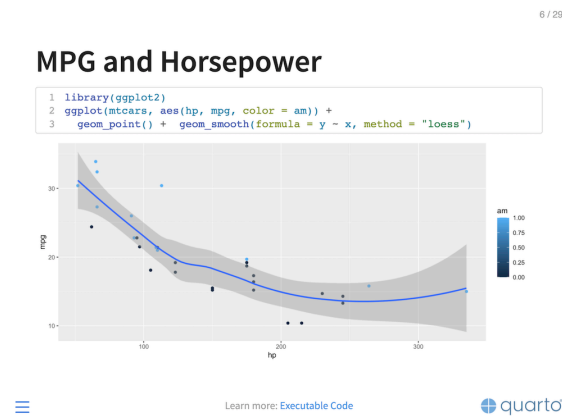
{r}
summary(cars)

## Second Slide

 $E = mc^2$ 
```

29.26.3. Quarto

Quarto is the official successor to R Markdown, developed by Posit (formerly RStudio). It supports R, Python, Julia, and Observable JS. It has **native** support for Typst as an output format since version 1.4, which makes it a natural bridge toward Typst.



Strengths

- Multi-language (R, Python, Julia)
- It's my favorite (does that count as a strength?)
- Native Typst output (v1.4+)
- Interactive slides with Reveal.js
- Modern, active ecosystem (very structured, and the ability to run so many different languages is incredible)

Limitations

- Abstraction that hides fine control
- Limited Typst templates compared to native
- Learning curve for the options

Quarto configuration with Typst output:

```
---
title: "Data Analysis"
format:
  typst:
    toc: true
    papersize: a4
    mainfont: "Linux Libertine"
    section-numbering: "1.1."
---
```

Section

Text and Python code:

```
{python}
import numpy as np
print(np.mean([1, 2, 3, 4, 5]))
```

29.26.4. knitr

knitr is the computation engine underlying R Markdown and Quarto. It runs the code blocks, captures the output (text, charts, tables), and inserts it into the document. With knitr you can generate Typst output directly from R.

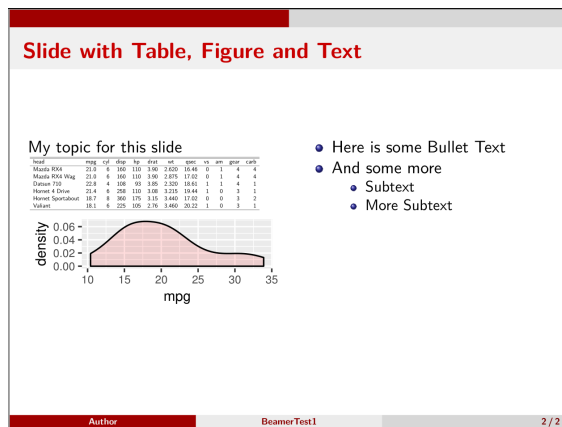


Figure 4: Example screenshot taken from [HERE](#).

Using knitr with Typst output:

```
library(knitr)

# Set the Typst output
opts_knit$set(out.format = "typst")

# Tables are converted automatically
knitr::kable(head(mtcars, 3), format = "pipe")

# With tinytable (native Typst output)
library(tinytable)
tt(head(iris, 5)) |>
  style_tt(i = 0,
    background = "#1a3a5c",
    color = "white")
```

29.26.5. Polylux

Polylux is the most widespread Typst package for presentations. It adds overlay management (progressive reveal of elements) and presenter mode to Typst's page system.

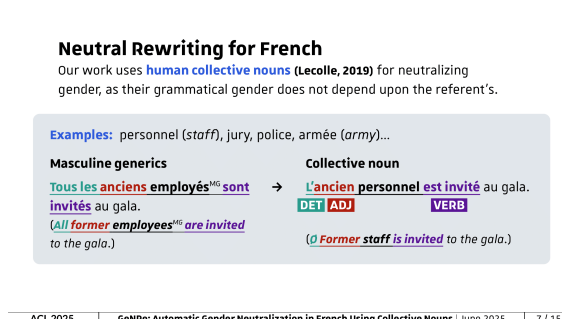


Figure 5: Example screenshot taken from [HERE](#).

Basic structure of a Polylux presentation:

Strengths

- Full integration with R
- Caching of computational results
- Typst output support (knitr ≥ 1.45)
- `tinytable` : R tables → native Typst

Limitations

- R only (not Python)
- Requires R installed
- R compilation latency

Strengths

- Native Typst syntax
- Simple overlays and pauses
- Completely free layout
- No predefined theme to learn

Limitations

- Limited predefined themes
- Less “magical” than Beamer
- Requires more configuration

```

#import "@preview/polylux:0.3.1": *

#set page(paper: "presentation-16-9")
#set text(font: "Linux Libertine", size: 22pt)

#polylux-slide[
  = Presentation Title

  #align(center + horizon)[
    Descriptive subtitle
    #v(1em)
    _Author · Institution · Year_
  ]
]

#polylux-slide[
  == Content with progressive reveal

  - First item always visible
  #pause
  - Second item appears on click
  #pause
  - Third item appears later
]

#polylux-slide[
  == Two columns

  #grid(columns: (1fr, 1fr), gutter: 2em,
    [*Left column*\ Text and content],
    [*Right column*\ Other content],
  )
]

```

29.26.6. Touying

Touying is the most advanced Typst framework for presentations. It offers professional predefined themes (Metropolis, University, Dewdrop, Stargazer...) and very sophisticated overlay management.

Outline

| | |
|----------------------|---|
| Example Slides | 2 |
| Bullet Points | 3 |
| A CeTZ Figure | 4 |
| References | 7 |

2025-10-08

1 / 8

Figure 6: Example screenshot taken from [HERE](#).

Strengths

- Mature, professional themes
- Advanced overlays (`#uncover` , `#only`)
- Automatic handout mode
- Built-in navigation bar

Limitations

- More complex API than Polylux
- Versions change often (check Universe)
- Requires more initial study

Touying setup with the University theme:

```
#import "@preview/touying:0.5.3": *
#import themes.university: *
```

```
#show: university-theme.with(
  aspect-ratio: "16-9",
  config-info(
    title: [The Presentation Title],
    subtitle: [Subtitle],
    author: [Mario Rossi],
    institution: [University of Milan],
    date: datetime.today(),
  ),
)
```

```
// Automatic title slide
#title-slide()
```

```
// Section
== First Section
```

```
== Slide with overlay
```

```
#pause
```

First element — appears immediately.

```
#pause
```

Second element — appears on the second click.

```
== Slide with uncover
```



```
#uncover("2-")[Visible from slide 2 onward]
#only(3)[Visible ONLY on slide 3]
```

29.27. Final comparison: which to choose?

| Tool | Ideal for | Curve | PDF quality | Code+Data |
|------------|-----------------------------------|--------|-------------|-----------|
| Beamer | Academic articles with slides | High | ★★★★★ | No |
| R Markdown | Data science, statistics | Medium | ★★★☆☆ | R |
| Quarto | Multi-language, reproducibility | Medium | ★★★★☆ | R/Python |
| knitr | Engine underlying Quarto/Rmd | Medium | ★★★★☆ | R |
| Polylux | Custom slides in Typst | Low | ★★★★★ | No |
| Touying | Academic slides with ready themes | Medium | ★★★★★ | No |

Tip

If you already know LaTeX and want maximum compatibility with journals and conferences → **Beamer**. If you work with R/Python data and want computational reproducibility → **Quarto** (with Typst output for quality). If you want total control over the design and you're already using Typst → **Touying** for ready themes, **Polylux** for maximum freedom.

In general choose the one that seems simplest for what you need to do; I love Quarto: it's nice, easy, fast (it has tons of [Documentation](#)) and the [Posit Cloud](#) platform is fantastic.

Appendix F (Part II) — Additional Notes: Programming and Automation

29.28. The Python `typst` package

The official `typst` package for Python lets you compile `.typ` documents directly from Python, without using the terminal or `subprocess` calls.

```
pip install typst
```

29.28.1. Compiling a file

```
import typst

# Compile an existing .typ file → PDF
typst.compile("document.typ", output="document.pdf")

# Compile from a Python string → PDF bytes in memory
source = """
= Title generated by Python

Today's date: #datetime.today().display()

- First item
- Second item
"""
pdf_bytes = typst.compile(source)

with open("output.pdf", "wb") as f:
    f.write(pdf_bytes)
```

29.28.2. Passing parameters with sys.inputs

`sys.inputs` lets you inject string values into the Typst document from the outside, without modifying the source file:

```
import typst

typst.compile(
    "template.typ",
    output="report-mario.pdf",
    sys_inputs={
        "author": "Mario Rossi",
        "title": "Quarterly Report",
        "date": "June 2026",
        "version": "2.1",
    },
)
```

In the `template.typ` file, read the values like this:

```
// Values always arrive as strings
#let author = sys.inputs.at("author", default: "Author")
#let title = sys.inputs.at("title", default: "Document")
#let date = sys.inputs.at("date", default: "")
#let version = sys.inputs.at("version", default: "1.0")

#set document(title: title, author: author)

≡ #title
```

Written by ***#author*** — **#date** — version **#version**

From the command line the equivalent mechanism is:

```
typst compile template.typ report.pdf \
--input author="Mario Rossi" \
--input title="Quarterly Report" \
--input date="June 2026"
```

29.29. Reading JSON data in the document

For complex data structures (tables, lists, metrics) it's more practical to pass a JSON file than to use `sys.inputs`:

```
// In the Typst document: reads data.json from the same folder
#let data = json("data.json")

// Access the fields like a normal dictionary
= #data.title

Author: #data.author — Date: #data.date

// Generate a table from the data
#table(
  columns: data.headers.len(),
  stroke: 0.6pt + luma(200),
  fill: (_, row) => if row == 0 { C.blu } else { white },
  table.header(
    ..data.headers.map(h =>
      table.cell[#set text(fill: white, weight: "bold"); #h]
    ),
  ),
  ..data.rows.map(r => r.map(c => [#c])).flatten(),
)
```

The corresponding `data.json` file:

```
{
  "title": "Sales Report — Q2 2026",
  "author": "Analytics Team",
  "date": "30 June 2026",
  "headers": ["Region", "Revenue", "Orders", "Trend"],
  "rows": [
    ["North", "€ 58,100", "312", "↑ +8%"],
    ["Center", "€ 41,200", "228", "↑ +15%"],
    ["South", "€ 43,000", "307", "↑ +13%"]
  ]
}
```

```
]
}
```

From Python, generate the JSON and compile:

```
import typst, json
from datetime import date

data = {
    "title": f"Sales Report — Q2 {date.today().year}",
    "author": "Analytics Team",
    "date": date.today().strftime("%d/%m/%Y"),
    "headers": ["Region", "Revenue", "Orders", "Trend"],
    "rows": [
        ["North", "€ 58,100", "312", "↑ +8%"],
        ["Center", "€ 41,200", "228", "↑ +15%"],
        ["South", "€ 43,000", "307", "↑ +13%"],
    ],
}

with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

typst.compile("template.typ", output="report.pdf")
print(" report.pdf generated!")
```

29.30. The Data → Template → PDF pattern

The fundamental flow for automatic document production:



Real use cases for this pattern:

- **Custom certificates:** 500 certificates from a CSV with names and dates — one template, one Python loop
- **Weekly reports:** data from a database → JSON → PDF emailed every Monday morning
- **Automatic invoices:** each order from the management system generates its PDF in a few milliseconds
- **University lecture notes:** instructors update the data in a Google Sheet → the PDF regenerates itself

29.31. Batch production: certificate example

```
import typst, csv, os

# certificate-template.typ must have:
# #let name = sys.inputs.at("name", default: "")
# #let course = sys.inputs.at("course", default: "")
# #let date = sys.inputs.at("date", default: "")

os.makedirs("certificates", exist_ok=True)

with open("participants.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        typst.compile(
            "certificate-template.typ",
            output=f"certificates/{row['id']}.pdf",
            sys_inputs={
                "name": row["name"],
                "course": row["course"],
                "date": row["date"],
            },
        )
        print(f"Certificate for {row['name']} generated.")

print("All certificates are ready!")
```

29.32. Integration with FastAPI

To generate PDFs on demand via a web API:

```
from fastapi import FastAPI
from fastapi.responses import Response
import typst, json, tempfile, os

app = FastAPI()

@app.post("/pdf/report",
    response_class=Response,
    responses={200: {"content": {"application/pdf": {}}}})
async def generate_report(data: dict):
    # Write the temporary JSON
    with tempfile.NamedTemporaryFile(
        mode="w", suffix=".json",
        delete=False, encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False)
        json_path = f.name
```

```
try:
    # Compile (typst reads the json from the same path
    # only if the template uses json("data.json") with an absolute path)
    pdf = typst.compile("report-template.typ")
    return Response(
        content=pdf,
        media_type="application/pdf",
        headers={"Content-Disposition":
            "attachment; filename=report.pdf"},
    )
finally:
    os.unlink(json_path)
```

29.33. GitHub Actions: automatic reports

```
# .github/workflows/report.yml
name: Weekly Report
on:
  schedule:
    - cron: "0 7 * * 1" # Every Monday at 7:00 UTC

jobs:
  generate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with: { python-version: "3.12" }

      - name: Setup Typst
        uses: typst-community/setup-typst@v3

      - name: Install dependencies
        run: pip install typst pandas

      - name: Generate data and PDF
        run: |
          python scripts/generate_data.py # creates data.json
          typst compile template.typ report.pdf

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: weekly-report
          path: report.pdf
```

29.34. Best practices for Typst in production

| Practice | Rationale |
|---|---|
| Separate data and template | The <code>.typ</code> doesn't change on every run; the data does. Keeping them separate simplifies maintenance and debugging. |
| Version templates with Git | If a template update breaks the PDFs, you can revert to the previous version in one command. |
| Specify the Typst version | Add <code>typst.toml</code> to the project with the exact version used, to ensure reproducibility over time. |
| Test with extreme data | Tables with 0 rows, very long names, special characters (accents, emoji): check that the template holds up. |
| Fonts included in the repository | If you use custom fonts, put them in the project folder to avoid differences between machines. |
| Use <code>default:</code> in <code>sys.inputs</code> | Every <code>sys.inputs.at()</code> should have a default value to avoid crashes if a parameter is missing. |

You've reached the end (finally)!

You now have many, if not all, the tools to create professional documents with Typst.

The best way to learn is obviously to **practice** and **practice**.
I hope this handbook has been useful to you.

For anything, let me know on my contacts page:

[STEFANO COELATI RAMA](#)